

Fine-Grained Non-Interactive Key Exchange without Idealized Assumptions, and Lower Bounds ^{*}

Yuyu Wang ¹ 

Chuanjie Su ¹ 

Jiaxin Pan ² 

Chunxiang Xu[†] ¹ 

¹ University of Electronic Science and Technology of China, Chengdu, China
wangyuyu@uestc.edu.cn, chuanjie.su@hotmail.com, chxxu@uestc.edu.cn

² University of Kassel, Kassel, Germany
jiaxin.pan@uni-kassel.de

Abstract

In this paper, we study multi-party non-interactive key exchange (NIKE) in the fine-grained setting. More precisely, we propose three multi-party NIKE schemes in three computation models, namely, the bounded-parallel-time, bounded-time, and bounded-storage models. The bounded-parallel-time and bounded-time constructions rely on mild fine-grained assumptions, whereas the bounded-storage construction is unconditional. This improves the recent work of Afshar, Couteau, Mahmoodi, and Sadeghi (EUROCRYPT 2023) that requires idealized assumptions, such as random oracles or generic groups.

Additionally, we show that all our constructions satisfy a natural desirable property that we refer to as extendability, and we give generic transformations from extendable multi-party NIKes to multi-party identity-based NIKes in the fine-grained setting.

Furthermore, we generalize the lower bound on users' storage consumption in the bounded-storage model by Dziembowski and Maurer (Eurocrypt 2004) to encompass any multi-party NIKE with extendability and local key independence. Together with our construction, this lower bound shows that the users' storage consumption of our multi-party NIKE in the bounded-storage model is optimal up to constant factors within this natural class.

Keywords: Multi-party non-interactive key exchange, fine-grained cryptography, complexity assumption, lower bound, identity-based cryptography, bit-fixing pseudorandom function.

1 Introduction

1.1 Background

Non-interactive key exchange (NIKE) is a central primitive in cryptography, enabling a group of parties to simultaneously release a single message (i.e., the public key), and then securely agree on a session key in the presence of any potential adversaries after observing the public keys. Existing two-party NIKE schemes are constructed based on various cryptographic assumptions. These include well-established assumptions such as the Diffie-Hellman assumption [2], Learning With Errors [4], and assumptions related to factoring [?], and more exotic ones such as the existence of generic groups [?, ?] and algebraic groups [?]. Joux [7] proposed a three-party NIKE based on the bilinear Diffie-Hellman assumption over pairings. When it comes to multi-party NIKE involving more than four parties, all known candidates (against polynomial-time adversaries) are in the world of “obfustopia”, where obfuscation exists [?]. This raises a natural question of whether it is possible to construct multi-party NIKE from much more established assumptions.

Multi-party NIKE based on mild assumptions. Very recently, Afshar et al. [?] provided an affirmative answer to the above question by introducing n_p -party NIKE schemes for any constant n_p

^{*}A preliminary version of this paper appeared at Crypto 2024 [?]; this is the full version.

[†]Corresponding author.

in the random oracle model and Shoup’s $((n_p/2 - 1)$ -linear) generic group model (i.e., the idealized multilinear map model where the number of groups is $n_p/2$). These schemes are in the context of *fine-grained cryptography* [?], where adversaries can only have limited resources and honest users do not possess more resources than adversaries. One of the main motivations of fine-grained cryptography is to base security on weak assumptions, or in some models no computational assumptions at all. Hence, the use of random oracles or generic groups makes their work less desirable.

Our Goal: Can we construct a fine-grained multi-party NIKE without idealized assumptions, such as random oracles and generic groups?

We take a closer look at the fine-grained setting. Essentially, there are different ways to restrict users’ and adversaries’ computing power. It can be in terms of running time (e.g., [10, ?, ?]), parallel running time (i.e., circuit depths) (e.g., [5, ?, ?]), or storage (e.g., [8, ?, ?]). The multi-party NIKE of Afshar et al. was treated in the bounded-time model. In their scheme, an honest user runs in time $O(\lambda^{n_p-1})$ (respectively, $O(\lambda)$) and the scheme is secure against adversaries running in time $o(\lambda^{n_p})$ in the random oracle model (respectively, $o(\lambda^2)$ in Shoup’s $(n_p/2 - 1)$ -linear generic group model). Although the assumptions of the random oracle and generic group model are already considered weak for multi-party NIKE schemes, they still fall into the worlds of “minicrypt” (the random oracle model), “cryptomania” (the generic group model), or even “obfustopia” (the multilinear generic group model) when dealing with a large number of parties. This raises the question of whether we can have multi-party NIKE even if we are in, say, the Pessiland, where one-way functions do not exist.

Concurrent work. An independent and concurrent recent work by Couteau et al. [?] introduced a 4-party NIKE protocol with a quadratic gap between users and adversaries (with honest users running in time $O(\lambda)$ and secure against adversaries running in time $o(\lambda^2)$), assuming the existence of exponentially secure injective pseudorandom generators (exp-iPRGs) and the sub-exponential hardness of the computational Diffie-Hellman problem (seCDH). Their results can also be extended to a 6-party NIKE under exp-iPRGs and CDH in bilinear groups, as well as to a 6-party NIKE (respectively, 9-party NIKE) with honest users running in time $O(\lambda)$ and secure against adversaries running in time $o(\lambda^{1.5})$ under exp-iPRGs and CDH without pairings (respectively, with pairings). They further showed that their 4-party protocol is secure in Maurer’s generic group model with a smaller gap between users and adversaries (with honest users running in time $O(\lambda)$ and secure against adversaries running in time $o(\lambda^{1.6})$), assuming the existence of non-uniformly exponentially secure injective pseudorandom generators.

1.2 Our Contributions

We revisit multi-party NIKE in the fine-grained setting and propose constructions in the bounded-parallel-time, bounded-time, and bounded-storage models based on weak fine-grained complexity assumptions or, in the BSM, no computational assumptions. More precisely:

- In the bounded-parallel-time model, we construct an n_p -party NIKE for any constant n_p computable in $AC^0[2]$ and secure against adversaries in NC^1 (i.e., circuits with logarithmic depth and polynomial size). Clearly, the honest users consume less computational resources than adversaries, since $AC^0[2] \subsetneq NC^1$ [11, ?]. Similar to prior works in the same setting (e.g., [?, ?]), our security is based on the worst-case assumption $NC^1 \subsetneq \oplus L / \text{poly}$, which is widely believed to hold. All existing NC^1 -fine-grained schemes are based on $NC^1 \subsetneq \oplus L / \text{poly}$ [?, ?, ?, ?, ?, ?]. According to Barrington, $\oplus L / \text{poly}$ is the class of languages with polynomial-sized branching programs and NC^1 languages have polynomial-sized branching programs of constant width [?]. Hence, this assumption holds if there exists one language having only branching programs of super-constant width. It would be quite surprising if this does not hold.
- In the bounded-time model, we construct an n_p -party NIKE for any constant n_p computable in $\tilde{O}(\lambda^{n_p+k-1})$ and secure against adversaries running in time $\tilde{O}(\lambda^{n_p+k-\Omega(1)})$. The underlying assumption is the average-case Zero- k -Clique hypothesis for any constant $k > 2$, which is exactly the one used in [?] for constructing fine-grained two-party NIKE. It is also widely used in the fine-grained complexity to derive conditional lower bounds for other problems (e.g., [?, ?, ?, 1]). Note that no known approach for this assumption is faster than essentially λ^k , and the worst-case Zero- k -Clique hypothesis for any constant k does not appear to imply $P \neq NP$ [?]. Moreover, although subsequent work [?] shows that the conjecture fails for sufficiently large k in the absence of one-way functions,

our construction already achieves the largest security gap at $k = 3$, which is the weakest instantiation of the assumption since its strength increases with k . Therefore, the large- k separation of [?] does not apply to our small-constant regime and does not imply that our (strictly weaker) assumption entails one-way functions.

- In the bounded-storage model (BSM) [?], we extend the two-party NIKE in [?] to an n_p -party NIKE for any constant n_p . The resulting scheme is unconditionally secure against adversaries with memory bounds $O(\lambda^{n_p+1})$ and each user consumes $O(\lambda^{n_p})$ bits of memory. Also, the total communication cost between the parties (i.e., the total size of the public keys in this case) is $O(\lambda)$. This is also part of our efficiency measure.

These results show that weak (non-idealized) fine-grained assumptions, and in the BSM even without any computational assumptions, suffice to realize multi-party NIKE, which was previously achieved only via obfuscation- or multilinear-map-based constructions against polynomial-time adversaries, or under idealized assumptions in the fine-grained setting.

Remarks on the uniformly random string (URS). While our construction in the bounded-time model is proposed in the plain model, where no trusted public parameter is exploited, the constructions in the bounded-parallel-time model and BSM mentioned above are instantiated in the URS model, where there exists a URS accessible to all parties and adversaries. It is important to note that this model is weaker than the common reference string model, since the URS is generated in a “nothing up my sleeve” manner, ensuring that it contains no hidden information or trapdoors. Moreover, our scheme in the bounded-parallel-time model can also be instantiated without using a URS in the random oracle model, by deriving the URS from a hash function modeled as a random oracle. The same argument does not apply to our construction in the BSM, since it crucially relies on the URS disappearing after a certain period of time.

Comparison. In Tables 1 and 2, we provide a comparison of all existing fine-grained multi-party NIKE constructions. The table contrasts the schemes with respect to the number of parties, adversarial and user complexity, the use of a uniform random string (URS), correctness error, security error, computational model, the underlying assumptions, and key lengths.

Scheme	#P	Adv. Comp.	User Comp.	URS	Correctness Error	Security Error	Model	Assumption
ACMS23[?]	n_p	$\lambda^{n_p/(n_p-1)}$	$\tilde{O}(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	ROM
	$2n_p$	$o(\lambda^2)$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	$(n_p - 1)$ -linear SGGM
BCS24[?]	4	$o(\lambda^2)$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	exp-iPRGs, seCDH
	6	$o(\lambda^2)$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	exp-iPRGs, seCDH2
	6	$o(\lambda^{1.5})$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	exp-iPRGs, seCDH
	9	$o(\lambda^{1.5})$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	exp-iPRGs, seCDH2
	4	$o(\lambda^{1.6})$	$O(\lambda)$	No	$\text{negl}(\lambda)$	$\text{negl}(\lambda)$	T	exp-iPRGs, MGGM
Section 3	n_p	NC^1	$\text{AC}^0[2]$	Yes	0	$\text{negl}(\lambda)$	P	$\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$
Section 4	n_p	$\tilde{O}(\lambda^{n_p+k-\Omega(1)})$	$\tilde{O}(\lambda^{n_p+k-1})$	No	$O(\lambda^{k+n_p-kn_p})$	$p_{\text{GL}}^{-1}(\epsilon_w)$	T	Zero- k -Clique
Section 5	n_p	$O(\lambda^{n_p+1})$	$O(\lambda^{n_p})$	Yes	$2/\lambda$	$O(1/\sqrt{\lambda}), \text{negl}(\lambda)$	M	None

Table 1: Comparison of fine-grained multi-party NIKES. Here, λ denotes the security parameter and n_p the (constant) number of parties. negl is an unspecified negligible function. Let $k \geq 3$ be a constant; larger k implies a stronger Zero- k -Clique assumption. The column “#P” denotes the number of parties, “Adv. Comp.” the complexity of the adversary, “User Comp.” the complexity of the user, and “URS” whether a uniform random string is used. In the “Model” column, “T”, “P”, and “M” denote the bounded-time, bounded-parallel-time, and bounded-storage models, “seCDH2” denotes the seCDH assumption with bilinear pairings. For Section 4, $\epsilon_w = 1/200 + \tilde{O}(\lambda^{-1}) + \tilde{O}(\lambda^{n_p+k-kn_p})$ denotes the weak-security error before the Goldreich–Levin transformation, and $p_{\text{GL}}(\epsilon) = \epsilon^{O(1)}$ is the standard Goldreich–Levin reconstruction success probability. For Section 5, the “Adv. Comp.” and “Security Error” columns denote that the construction satisfies the $(O(\lambda^{n_p+1}), O(1/\sqrt{\lambda}), \text{negl}(\lambda))$ -security in the BSM as defined in Section 2.1.

Note that although our schemes in the bounded-time model and the BSM incur non-negligible correctness and security errors, similar to the 2-party fine-grained NIKES in [?, ?] under the same assumptions and with comparable asymptotic parameters, these errors remain acceptable for concrete parameter choices in practice (see the concrete examples in Sections 4 and 5).

Extension to multi-party identity-based NIKE (IB-NIKE). We further show that our NIKES

Scheme	#P	Key	PK	SK
ACMS23[?]	n_p	$O(\log \lambda)$	$O(\lambda^{n_p/(n_p-1)})$	$O(\lambda \log \lambda)$
	$2n_p$	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
BCS24[?]	4	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
	6	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
	6	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
	9	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
	4	$O(\log \lambda)$	$O(\lambda \log \lambda)$	$O(\lambda \log \lambda)$
Section 3	n_p	$\text{poly}(\lambda)$	$\text{poly}(\lambda)$	$\text{poly}(\lambda)$
Section 4 (TNIKE*)	n_p	1	$\tilde{O}(\lambda^{(n_p+2)/(n_p+k-1)})$	$\tilde{O}(\lambda^{(n_p-1)/(n_p+k-1)})$
Section 5	n_p	$O(\lambda)$	$O(\lambda)$	$O(\lambda^{n_p})$

Table 2: Comparison of fine-grained multi-party NIKEs. Here, λ denotes the security parameter. Let $k \geq 3$ be a constant in the Zero- k -Clique assumption. The column “|Key|” denotes shared key length, “|PK|” denotes the public key length, and “|SK|” denotes the secret key length. For Section 4, the row reports the Goldreich–Levin-transformed one-bit scheme TNIKE*.

$$\text{Multi-Party IB-NIKE} \xleftarrow{\text{Sec. A.2}} (\text{Restricted}) \text{BPRF} \xleftarrow{\text{Sec. A.3}} \text{Extendable Multi-Party NIKE}$$

Figure 1: Overview of our approaches in constructing IB-NIKE in the fine-grained setting.

can be transformed into IB-NIKEs for any constant number of parties. Notice that such IB-NIKEs can be trivially constructed from NIKE by including NIKE public keys of all users in the IB-NIKE key for each identity. However, our construction of IB-NIKE is significantly better than this trivial one, since the identity-space of IB-NIKE is much larger. If the underlying NIKE supports a super-constant (say, polynomial) number of users, then our IB-NIKEs can support an exponentially large identity-space. Indeed, for our results in the BSM, making the number of users super-constant does not spoil the security proof since it is based on no assumptions. Similarly, our results in the bounded-time model can also support a super-constant number of users if we make the underlying assumption stronger, i.e., Zero- k -Clique assumption for super-constant k , which, however, may imply $P \neq NP$ [?].

Our transformation contains several intermediate steps, as described figuratively in Figure 1. Specifically, we follow the technique in [6] to show that in the fine-grained setting, bit-fixing pseudorandom functions (BPRFs) can be converted into multi-party IB-NIKEs, and then provide a generic construction of BPRF from any NIKE with a natural property we call extendability, which is satisfied by all our constructions (and indeed also the ones in previous works [7, ?]) except for those based on multilinear maps. Here we note that our multi-party NIKE in the bounded-parallel-time model only satisfies restricted extendability and can only be converted into a restricted version of BPRF. Nonetheless, this restricted version is sufficient for achieving multi-party IB-NIKE. Moreover, when the number of users is constant, instantiating the above transformation with any of our NIKE constructions preserves all parameters up to asymptotic equivalence. In particular, the adversarys resources, user complexity, and key length remain asymptotically the same as in the underlying NIKE. We refer the reader to Section A for the full details.

Negative result in the BSM. In the BSM, in addition to our positive result, we extend the negative result in [?] for two-party NIKEs to show that for an n_p -party NIKE with (even restricted) extendability and a natural property we call local key independence, the entropy of the shared key conditioned on the public keys is bounded by roughly $m_u^{n_p}/m_a^{n_p-1}$, where m_u and m_a represent the amounts of memory consumed by each user and adversary respectively. Hence, for sharing λ bits, $m_u \gtrsim \sqrt[n_p]{\lambda \cdot m_a^{n_p-1}}$. Local key independence is the BSM-specific regularity condition formalized in Theorem 2.4. Here local keys are the intermediate keys produced during the extendability procedure. Roughly, it says that public keys and auxiliary key pairs leak only limited information about local keys, and that adding one more independently generated key pair does not create much extra leakage through the common URS. Additionally, as mentioned earlier, (restricted) extendability is satisfied by all existing multi-party NIKE schemes (except for those based on multilinear maps) to the best of our knowledge. Both properties are quite natural, and it would be surprising if any construction could circumvent our lower bound. This suggests that our multi-party NIKE in the BSM is optimal up to constant factors within the natural class of schemes satisfying extendability and local key independence.

As far as we know, this is the first negative result for multi-party NIKE in the BSM. In contrast to [?], our analysis does not require the schemes to satisfy perfect security. In this sense, our result applies

more broadly even in the two-party setting.

Practical implications. Other than relying on weak assumptions and having low computational complexity, our results have the following practical implications.

Our constructions in the bounded (parallel-)time setting are particularly well-suited for systems in which attacks are only meaningful if they succeed within a short time window. For example, users may employ the shared key to protect information that is valuable only temporarily and can be safely revealed or discarded afterward. Concretely, the key can be used to transmit short-lived information (e.g., verification codes) that are revealed only after a prescribed deadline. Similarly, the key can be used to encrypt files that, although logically deleted by users, remain temporarily retained by the cloud provider before permanent erasure. Security is required only during this enforced retention window.

Another direct application is the generation of message authentication codes (MACs) using the shared key. The verifier imposes a strict upper bound on the allowed computation time for generating a valid tag, and any party exceeding this bound is rejected. Thus, adversaries who require asymptotically more time gain no advantage, since tags produced after the deadline are invalid.

To illustrate the concrete magnitude of this gap, in our construction in the bounded-parallel-time model, the honest user runs in AC^0 and hence in NC^1 , whereas the known distinguishing approach for $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$ proceeds via determinant computation in NC^2 for matrices from ZeroSamp_λ and OneSamp_λ . For $\lambda = 2^{50}$ and any constant n_p , honest evaluation has depth $O(\log \lambda)$, which is about 50 ignoring constant factors (and can in fact be implemented in $\text{AC}^0[2]$ using only a constant number of layers). In contrast, this determinant-based approach has depth $O((\log \lambda)^2)$, which is about 2500 ignoring constant factors. This concrete separation makes the timeout mechanism practically enforceable.

In the bounded-time model, although the security gap is smaller, the same paradigm applies. For example, when the number of parties is 3, each honest party runs in time about $\tilde{O}(\lambda^5)$, whereas the scheme remains secure against adversaries below the λ^6 barrier by a polynomial factor. By enforcing a strict sequential time budget for generating valid MACs, the system ensures that honest users succeed within the permitted window, while slower adversaries are rejected.

In the BSM, our construction is optimal up to constant factors within the natural class of schemes satisfying extendability and local key independence, as shown by our matching impossibility result. This setting naturally captures scenarios where a high-speed broadcast channel generates URSs that are too large for an adversary to store but can be processed on the fly by honest parties.

More generally, as observed in [?], combining fine-grained primitives with standard cryptographic primitives yields hybrid schemes that are secure against restricted adversaries under weak (or even no) assumptions, while retaining security against polynomial-time adversaries under stronger assumptions.

1.3 Technical Details

Multi-party NIKE based on $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$. We now give technical details on how we construct our fine-grained multi-party NIKE in the bounded-parallel-time model. All the adversaries we consider in this part are in NC^1 . Similar to previous works [?, ?, ?, ?, ?, ?], we exploit the indistinguishability of two specific distributions ZeroSamp_λ and OneSamp_λ outputting $\lambda \times \lambda$ matrices with rank $\lambda - 1$ and full rank respectively. It is implied by $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$.

Our starting point is to utilize the fine-grained hash proof system proposed in [?] to construct an NIKE via the technique proposed in [?]. Roughly we can let the projection and the statement of the hash proof system serve as the public keys in a two-party NIKE and let the parties share the proof with the (secret) verification key and the witness respectively. Security is guaranteed by the smoothness of the hash proof system. To extend it to a multi-party one, the first problem we must address is that the key pairs are vectors generated by different types of algorithms. When more parties are involved, it is unclear how to combine multiple vectors to generate a session key.

A straw-man solution is to extend the vectors to matrices. Taking the three-party case as an example, the public keys of the parties P_1 , P_2 , and P_3 are generated as $\mathbf{R}_1\mathbf{M}$, $\mathbf{M}\mathbf{R}_2$, and $\mathbf{M}\mathbf{R}_3$ respectively, where $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$ is the public parameter and $\mathbf{R}_i$'s are the random secret matrices in $\{0, 1\}^{\lambda \times \lambda}$. The parties can share $\mathbf{R}_1\mathbf{M}\mathbf{R}_2\mathbf{M}\mathbf{R}_3$ with certain entropy. However, unfortunately, it does not satisfy correctness. Specifically, $\mathbf{R}_1\mathbf{M}\mathbf{R}_2\mathbf{M}\mathbf{R}_3$ is indeed computable by P_1 and P_2 with their secret matrices, while it is not computable by P_3 due to the lack of knowledge on $\mathbf{R}_2\mathbf{M}$. To address this issue, we

need to additionally publish $\mathbf{R}_2\mathbf{M}$. Nevertheless, this change will result in too much leakage about $\mathbf{R}_2$, compromising the security proof.

To overcome the technical hurdles mentioned above, we exploit the power of symmetric matrices. Specifically, we make the public parameter symmetric by computing $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$, and generate $\widetilde{\mathbf{M}}\mathbf{R}_i$ and $\mathbf{R}_i$ as the public/secret key pair for each party, where $\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$ and SymR_λ is the uniform distribution over symmetric matrices in $\{0, 1\}^{\lambda \times \lambda}$. The shared key is in the form of $\mathbf{R}_1 \widetilde{\mathbf{M}} \mathbf{R}_2 \cdots \widetilde{\mathbf{M}} \mathbf{R}_n$. Thanks to the symmetry of the matrices, each party knows both $\widetilde{\mathbf{M}}\mathbf{R}_i$ and $\mathbf{R}_i \widetilde{\mathbf{M}} = \mathbf{R}_i^\top \widetilde{\mathbf{M}}^\top = (\widetilde{\mathbf{M}}\mathbf{R}_i)^\top$, enabling each party to compute the shared key, ensuring correctness.

Since the secret keys are not uniformly random in $\{0, 1\}^{\lambda \times \lambda}$ now, we can no longer rely on the smoothness of the fine-grained hash proof system to prove security. To find a counterpart compatible with symmetric matrices, we define two new distributions $\mathcal{D}_0$ and $\mathcal{D}_1$:

$$\underbrace{\{\widetilde{\mathbf{M}}\mathbf{R} : \mathbf{R} \xleftarrow{\$} \text{SymR}_\lambda\}}_{=\mathcal{D}_0} \text{ and } \underbrace{\{\widetilde{\mathbf{M}}\mathbf{R} + \mathbf{I}_\lambda : \mathbf{R} \xleftarrow{\$} \text{SymR}_\lambda\}}_{=\mathcal{D}_1},$$

where $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$ for the public parameter $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$ and $\mathbf{I}_\lambda$ is the identity matrix. We then prove the indistinguishability of $\mathcal{D}_0$ and $\mathcal{D}_1$ by proposing a new technique to leverage the symmetry and the indistinguishability of the matrices. For more details, we refer the reader to Section 3.

A useful property of the above distribution is that the last column and row vectors in a matrix in $\mathcal{D}_0$ (respectively, $\mathcal{D}_1$) must be in (respectively, outside) the span of $\widetilde{\mathbf{M}}$. Hence, by embedding a random bit into the secret key of one party via vectors in $\text{Ker}(\widetilde{\mathbf{M}})$, and therefore in $\text{Ker}(\mathbf{M})$, and switching the distributions of public keys of other parties between $\mathcal{D}_0$ and $\mathcal{D}_1$ in a flexible way, we can prove that the right-bottom entry of the shared matrix is uniformly random in the view of an NC^1 adversary. By running the scheme multiple times in parallel, they can share a session key with any polynomial size, without increasing the complexity of the circuits' depth.

We note that [?] has shown the existence of a public-coin algorithm (denoted by RandSamp_λ in this work, see Figure 3) with the output distribution being identical to ZeroSamp_λ and OneSamp_λ with probability one half each. Therefore, we can use this algorithm to generate a public parameter indistinguishable from the original one, resulting in a fine-grained multi-party NIKE in the URS model.

Multi-party NIKE based on Zero- k -Clique. Our construction in the bounded-time model, which is based on the Zero- k -Clique hypothesis, extends the one presented in [?]. In the construction in [?], two parties exchange lists containing ℓ instances of the Zero- k -Clique problem, out of which $\sqrt{\ell}$ instances have solutions. The expectation is that there would be exactly a single instance in common having a solution, allowing the parties to share the index of this instance by brute forcing among the $\sqrt{\ell}$ instances. The Zero- k -Clique hypothesis says that finding the correct solution among the ℓ instances requires essentially solving all instances, ensuring security.

To extend this construction to a multi-party setting, a natural approach is to let all parties exchange their lists of instances, and share the common indices of the instances with one solution. The primary challenge lies in the rapid increase in the error probability of correctness as the number of users increases. To address this issue, we increase the number of instances with solutions for each party and allow users to share the sum of the overlapping part. For a rigorous analysis of correctness, we exploit a set of indicators in a careful manner. Each indicator, denoted by index i , equals 1 if the i th random index chosen by the last party is also chosen by all other parties, and 0 otherwise. Without loss of generality, we assume that the indices chosen by the last party are genuinely independent, while those chosen by each of the other parties are independent but conditioned on the event that they are all distinct. The former guarantees the independence of these indicators, and the latter ensures that each indicator equals 1 with a high probability. By computing the probability that all indicators are equal to 0, we obtain the small error probability.

Our reduction of security aims to find the correct solution among ℓ instances of the Zero- k -Clique problem by making use of an adversary that can recover the shared key. To simulate the view of the adversary, which consists of the transcript (i.e., the public keys) among different parties, the reduction splits the list into multiple ones with a solution in the same location, and plants $\ell^{(n_p-1)/n_p}$ solutions into the list. Then the reduction can extract the index of the solution by using the common indices in the shared key. Specifically, our security proof introduces a generalized splitting algorithm with a binary tree structure, which splits one instance into multiple ones having the same number of solutions as the split one.

By introducing additional checking procedures the proof can proceed without security loss potentially caused by the generalized splitting algorithm. When employing the Goldreich–Levin extractor [3] for privacy amplification, letting each party compress the shared indices by summing them up, as mentioned earlier, ensures that the running time of the Goldreich–Levin reduction remains acceptable.

To ensure a sufficient gap between users and adversaries, many more details need to be considered, including the number of solutions to be planted into instances during the simulation. We omit them here and refer the reader to Section 4.

Multi-party key exchange in the bounded-storage model. Recall that in the BSM, all parties and the adversary have access to a long URS. When the URS becomes unavailable, all parties broadcast public keys and share a session key with no additional interaction. The state-of-the-art NIKE (with public discussion) in this model is the two-party construction proposed in [?]. Each party stores a set of bits in the URS, and these bits are indexed in a way that they are (approximately) pairwise independent. Once the URS disappears, the parties exchange the indices and save the bits in common. It is crucial that the indices in the intersection are also pairwise independent so that the shared bits have high entropy. This allows the parties to apply privacy amplification. To ensure efficiency in terms of memory and communication, the parties utilize strongly 2-universal hash functions to represent the pairwise independent indices.

We apply the security analysis to the multi-user setting by proving that the common indices obtained from several approximately pairwise-independent index sets still satisfy the pairwise distribution needed for privacy amplification. While this is a natural extension of the two-party analysis, it has not been rigorously treated before, as far as we know. For correctness, we also need to show that the intersection contains sufficiently many distinct indices. Even in the two-party construction of [?], only the expected intersection size was analyzed. We prove a concentration bound by introducing indicators for the indices chosen by the last party. These indicators are not independent, while they are pairwise negatively correlated, which is enough for Chebyshev-type bound. The injectivity of the hash family ensures that their sum counts distinct common indices, and thus the proof analyzes the actual shared bits rather than multiplicity hits.

We would like to highlight that in [?], the Chernoff bound is used in an elegant way for proving the correctness of a fine-grained multi-party NIKE. Compared to their approach, ours offers conceptual simplicity by defining the indices differently, eliminating the need for a chain of Chernoff bounds, and is more general, in the sense that it works for pairwise-independent indices rather than only for completely independent ones.

Extendability and IB-NIKE. Without considering the steps for privacy amplification, the sharing procedures of all our constructions (as well as the multi-party NIKes in [?]) involve combining the secret key with public keys of other users sequentially. Each intermediate value, termed as a “local key”, essentially represents a shared key (without privacy amplification) amongst a subgroup of all parties. We refer to this inherent property as extendability. With this property, one can easily extend the shared (local) key amongst a certain group of parties to a larger group by simply using the public keys of newly joined parties, without the need to re-run the entire sharing procedure. Besides its independent interest, we observe that such a property provides us with a “multilinear map-like” structure: the public keys can be viewed as group elements, and together with a local key, they form a “multilinear Diffie-Hellman-like” tuple. By carefully combining this insight with the technique introduced by Boneh and Waters [?], which constructs BPRFs using multilinear maps, we obtain a generic construction of BPRFs from multi-party NIKes with extendability. Utilizing existing generic techniques [?, 6], we subsequently achieve fine-grained multi-party IB-NIKes from BPRFs.

Negative results on multi-party NIKE in the BSM. For adversaries with memory bound m_a , each user in our n_p -party NIKE in the BSM consumes $m_u = O(\sqrt{\lambda} \cdot m_a^{n_p-1})$ bits of memory, which might seem a bit disappointing. However, we extend the negative result of Dziembowski and Maurer [?] for two-party NIKes and show that this memory consumption is optimal up to constant factors within the natural class of multi-party NIKes satisfying extendability and local key independence.

The proof proceeds by induction. First, the entropy of the local key shared by two parties is bounded by roughly m_u^2/m_a . Then, assuming that the entropy of the local key shared by $i-1$ parties is at most roughly m_u^{i-1}/m_a^{i-2} , we show that the entropy of the local key shared by i parties is at most roughly m_u^i/m_a^{i-1} . The key observation is that extending an $(i-1)$ -party local key using the public key of a

new user can be viewed as a two-party key agreement step between the subgroup and the new user.

In the original two-party argument of [?], the parties' stored information is almost independent of Eve's view. This is no longer automatic when one party's state is replaced by an intermediate local key, which may already have some leakage. We handle this using local key independence, which bounds both the leakage of local keys and the extra leakage caused by the common URS. Together with the Dziembowski-Maurer averaging lemma [?] and Maurer's public-discussion bound [9], this yields the desired iterative lower bound. We refer the reader to Section 5.2 for the full details.

1.4 Comparison with the Proceedings Version

This is the extended version of the paper that appeared at Crypto 2024 [?]. In this version, we show new limitations of multi-party NIKes in the BSM, which imply the optimality of our construction up to constant factors within the natural class of schemes satisfying extendability and local key independence, in Section 5.2. Additionally, we give definitions, constructions, and security proofs of the fine-grained BPRFs and multi-party IB-NIKes in Section A.

2 Preliminaries

Notations. In this paper, algorithms are modeled as *functions* or *(streaming) word-RAMs*. A function represents a circuit with a specified domain and range, and we measure its complexity by its depth (i.e., the parallel running time). A word-RAM has access to memory and registers, each holding one word. Its running time is measured in the number of basic operations (e.g., addition, subtraction, multiplication, bit-shifting, and memory access). A word-RAM is a streaming one if it receives inputs in a streaming manner and its local memory is measured in bits.

We write $a \xleftarrow{\mathcal{S}} \mathcal{A}(b)$ (respectively, $a = \mathcal{A}(b)$) to denote the random variable output by a probabilistic (respectively, deterministic) algorithm $\mathcal{A}$ on input b . By $x \xleftarrow{\mathcal{S}}$ we denote the process of sampling an element x from a set or distribution $\mathcal{S}$ uniformly at random. By **negl** we denote an unspecified negligible function.

By $\mathbf{x} \in \{0, 1\}^n$ we denote a column vector with size n , by x_i we denote the i th element of a vector $\mathbf{x}$. By $[n]$ we denote the set $\{1, \dots, n\}$, by $[t_1, t_2]$ we denote the set $\{t_1, t_1 + 1, \dots, t_2\}$, and by $\text{MSB}_k(\mathbf{x})$ we denote the most significant k bits in (the bit string representing) $\mathbf{x}$. By $\log n$ we mean $\lceil \log n \rceil$. For some set or sequence $\mathcal{S}$, by $\mathbf{x}_{\mathcal{S}}$ we denote the sequence of all bits in $\mathbf{x}$ with indices (ordered lexicographically) in $\mathcal{S}$. For a matrix $\mathbf{A}$, we denote the set $\{\mathbf{x} \mid \mathbf{A}\mathbf{x} = \mathbf{0}\}$ by $\text{Ker}(\mathbf{A})$. By $\mathbf{I}_n$ we denote an identity matrix in $\{0, 1\}^{n \times n}$. By $\mathbf{0}$ we denote a zero vector or matrix.

By $H(x)$ we denote the *Shannon entropy* of a random variable x with alphabet $\mathcal{X}$ defined as

$$H(x) = - \sum_{\hat{x} \in \mathcal{X}} \Pr[x = \hat{x}] \log \Pr[x = \hat{x}].$$

By $H(x \mid y)$ we denote the *conditional entropy* of x conditioned on a random variable y with alphabet $\mathcal{Y}$ defined as

$$H(x \mid y) = \sum_{\hat{y} \in \mathcal{Y}} \Pr[y = \hat{y}] \cdot H(x \mid y = \hat{y}).$$

By $I(x; y)$ we denote the *mutual information* of x and y defined as

$$I(x; y) = H(x) - H(x \mid y),$$

which is the reduction of the uncertainty of x when y is learned. By $\mathbb{E}[x]$ and $\text{Var}[x]$ we denote the *expected value* and *variance* of the variable x . By $\Delta(x, y)$ we denote the *total variation distance* of two random variables x, y with alphabets $\mathcal{X}$ and $\mathcal{Y}$ defined as

$$\Delta(x, y) = (1/2) \sum_{\hat{x} \in \mathcal{X} \cup \mathcal{Y}} |\Pr[x = \hat{x}] - \Pr[y = \hat{x}]|.$$

A sequence of random variables $x_1, \dots, x_q$ with alphabet $\mathcal{X}$ is said to be *independent* if for any $\hat{x}_1, \dots, \hat{x}_q \in \mathcal{X}$, we have

$$\Pr[\wedge_{i=1}^q x_i = \hat{x}_i] = \prod_{i=1}^q \Pr[x_i = \hat{x}_i].$$

If we only require that

$$\Pr[x_i = \hat{x}_i \wedge x_j = \hat{x}_j] = \Pr[x_i = \hat{x}_i] \cdot \Pr[x_j = \hat{x}_j]$$

holds for any $1 \leq i < j \leq q$, then the sequence is said to be *pairwise independent*.

2.1 Fine-Grained Multi-Party NIKE

In this section, we define fine-grained multi-party NIKE and multi-party key exchange in the BSM. In the following, a circuit (respectively, word-RAM) family is a family of (possibly randomized) circuits (respectively, word-RAMs) with respect to all security parameters $\lambda \in \mathbb{N}$, denoted by $\{f_\lambda\}_{\lambda \in \mathbb{N}}$. For brevity, we refer to it as $\{f_\lambda\}$.

We now define the multi-party NIKE and its security. We require that the users be in $\mathcal{C}_1$ and the adversaries be in $\mathcal{C}_2$. In this work, $\mathcal{C}_1$ and $\mathcal{C}_2$ represent classes of circuit families in the bounded-parallel-time model, and represent classes of word-RAM families in the bounded-time model. In the BSM, $\mathcal{C}_1$ refers to a class of streaming word-RAM families with bounded storage, which receive inputs in a streaming manner and store the outputs.

Definition 2.1 (Multi-party non-interactive key exchange). A $\mathcal{C}_1$ - n_p -party non-interactive key exchange scheme (NIKE) consists of two algorithm families $\text{NIKE} = \{\text{Gen}_\lambda, \text{Share}_\lambda\} \in \mathcal{C}_1$ such that

- Gen_λ on input $\text{urs} \in \{0, 1\}^n$ for some $n = n(\lambda)$ outputs a public key pk and a secret key sk .
- Share_λ is a deterministic algorithm that on input a set of public keys $(\text{pk}_i)_{i \in [n_p]}$, an index i , and a secret key sk_i outputs either $\text{key} \in \mathcal{K}_\lambda$ or the abort symbol $\perp \notin \mathcal{K}_\lambda$, where $i \in [n_p]$ and $\mathcal{K}_\lambda$ is the session key space.

ϵ -correctness is satisfied if for all $\lambda \in \mathbb{N}$, all $i, j \in [n_p]$, we have

$$\Pr[\exists \text{key} \in \mathcal{K}_\lambda : \text{Share}_\lambda(i, \text{sk}_i, (\text{pk}_i)_{i \in [n_p]}) = \text{Share}_\lambda(j, \text{sk}_j, (\text{pk}_i)_{i \in [n_p]}) = \text{key}] \geq 1 - \epsilon$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$ and $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [n_p]$.

$\mathcal{C}_2$ - ϵ -security (respectively, $\mathcal{C}_2$ - ϵ -weak security) is satisfied if for any adversary $\mathcal{A} = \{\text{adv}_\lambda\} \in \mathcal{C}_2$ and all sufficiently large $\lambda \in \mathbb{N}$, we have

$$|\Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, (\text{pk}_i)_{i \in [n_p]}, \text{key})] - \Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, (\text{pk}_i)_{i \in [n_p]}, \text{key}')]| \leq \epsilon$$

$$(\text{respectively, } \Pr[\text{key} \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, (\text{pk}_i)_{i \in [n_p]})] \leq \epsilon)$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [n_p]$, $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$, and $\text{key}' \xleftarrow{\$} \mathcal{K}_\lambda$. In the plain model, we use the convention $\text{urs} = \perp$. If Share_λ outputs $\perp$ in a security experiment, the actual value $\perp$ is given to the adversary; thus abort probabilities are accounted for by the corresponding correctness and security bounds.

$(\mathbf{m}_a, \epsilon, \theta)$ -security in the BSM is satisfied if for any function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{\mathbf{m}_a}$, we have

$$\exists E : \Pr[E] \geq 1 - \epsilon \text{ and } I(\text{key}; (f(\text{urs}), (\text{pk}_i)_{i \in [n_p]})) \mid E \leq \theta,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [n_p]$, and $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$.

Remarks on the BSM. Here we note that the security in the BSM captures the situation where all parties have access to urs and the adversary is allowed to apply any (potentially unbounded) storage function f with an output length limit of $\mathbf{m}_a$ and store information on urs . Afterwards, urs becomes inaccessible, and all parties broadcast public keys, which are typically required to be short, and share a session key in a non-interactive way. Notice that we consider the traditional BSM [?] rather than the streaming BSM proposed by Dodis, Quach, and Wichs [?]. In the streaming BSM, the parties are allowed to communicate interactively with perhaps multiple rounds of long messages.

Extendability of Multi-Party NIKE. We now define a property of multi-party NIKE called extendability. Roughly, the extendability of a multi-party NIKE says that the sharing procedure involves an algorithm termed as Combine_λ , which sequentially combines a user's secret key with other public keys, and the intermediate value produced by Combine_λ , referred to as a local key, can be used to derive the shared key by using another algorithm Extract_λ .

Definition 2.2 (Extendability). A $\mathcal{C}_1$ - n_p -party non-interactive key exchange scheme $\text{NIKE} = \{\text{Gen}_\lambda, \text{Share}_\lambda\} \in \mathcal{C}_1$ satisfies ϵ -*extendability* if there exist two deterministic algorithm families $\{\text{Combine}_\lambda, \text{Extract}_\lambda\} \in \mathcal{C}_1$ such that, for every $\lambda \in \mathbb{N}$, the following holds. Let

$$\text{urs} \xleftarrow{\$} \{0, 1\}^n \text{ and } (\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs}) \text{ for all } i \in [n_p].$$

There exists an event E with $\Pr[E] \geq 1 - \epsilon$ such that, conditioned on E , for every $\mathcal{V} \subseteq [n_p]$, nonempty $\mathcal{V}' \subseteq \mathcal{V}$, $j \in \mathcal{V}$, and $i' \in \mathcal{V}'$, we have

$$\text{Combine}_\lambda(\text{localkey}_{\mathcal{V}'}, \{\text{pk}_i\}_{i \in \mathcal{V} \setminus \mathcal{V}'}) = \text{Combine}_\lambda(\text{sk}_j, \{\text{pk}_i\}_{i \in \mathcal{V} \setminus \{j\}})$$

and

$$\text{Extract}_\lambda(\text{localkey}_{[n_p]}, \text{pk}_{n_p}) = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]}),$$

where $\text{localkey}_{\mathcal{V}'} = \text{Combine}_\lambda(\text{sk}_{i'}, \{\text{pk}_i\}_{i \in \mathcal{V}' \setminus \{i'\}})$ and $\text{localkey}_{[n_p]} = \text{Combine}_\lambda(\text{sk}_1, \{\text{pk}_i\}_{i > 1})$.

We also give the definition of restricted extendability, which requires that Combine_λ only generates the local key of the users with consecutive indices.

Definition 2.3 (Restricted extendability). ϵ -*restricted extendability* is defined in exactly the same way as ϵ -extendability except that the set $\mathcal{V} \subseteq [n_p]$ is restricted to $\mathcal{V} = [t_1, t_2] \subseteq [n_p]$ for any $t_1, t_2 \in [n_p]$ and $\mathcal{V}'$ is restricted to $\mathcal{V}' = [s_1, s_2]$ for any $s_1, s_2 \in [n_p]$ such that $t_2 \geq s_2 \geq s_1 \geq t_1$.

Local key independence in the BSM. Next we define a natural property called local key independence for NIKE in the BSM. Roughly, it says that public information and auxiliary key pairs reveal only limited information about local keys, and that adding one more independently generated key pair reveals only limited additional information through the common URS.

Definition 2.4 (Local key independence). A n_p -party NIKE in the BSM with ϵ -(restricted) extendability satisfies $((\mu_i)_{i \in [n_p]}, t)$ -*local key independence in the BSM*, for $t \in \mathbb{N}$, if the following holds. In the key-generation experiment of Theorem 2.2 (or Theorem 2.3), let E be the corresponding extendability event. Conditioned on the same urs , independently sample $(\text{pk}'_j, \text{sk}'_j) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for $j \in [t]$ and set $\text{localkey}_{[i]} = \text{Combine}_\lambda(\text{sk}_1, (\text{pk}_j)_{2 \leq j \leq i})$. Then, for every $i \in [n_p]$, we have

$$I(\text{localkey}_{[i]}; (\text{pk}_j)_{j \in [i]}, (\text{pk}'_j, \text{sk}'_j)_{j \in [t]} \mid E) \leq \mu_i,$$

and for every $i \in [n_p]$ with $i > 1$, we have

$$\begin{aligned} & I(\text{localkey}_{[i-1]}; (\text{pk}_i, \text{sk}_i) \mid (\text{pk}_j)_{j \in [i-1]}, (\text{pk}'_j, \text{sk}'_j)_{j \in [t]}, E) \\ & \leq I(\text{localkey}_{[i-1]}; (\text{pk}_i, \text{sk}_i) \mid (\text{pk}_j)_{j \in [i-1]}, (\text{pk}'_j, \text{sk}'_j)_{j \in [t]}, \text{urs}, E) + \mu_i. \end{aligned}$$

3 Fine-Grained Multi-Party NIKE based on $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$

In this section, we propose a fine-grained multi-party NIKE running in $\text{AC}^0[2]$ and secure against NC^1 based on $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$. All arithmetic computations are over $GF(2)$ in this section.

3.1 Background on the NC^1 -Fine-Grained Setting

We recall and define sampling procedures in NC^1 , along with several lemmata.

Function families. We now recall the definitions of NC^1 circuits, $\text{AC}^0[2]$ circuits, and $\oplus\text{L}/\text{poly}$ circuits. Note that $\text{AC}^0[2] \subsetneq \text{NC}^1$ [11, ?].

Definition 3.1 (NC^1). The class of (non-uniform) NC^1 *function families* is the set of all function families $\mathcal{F} = \{f_\lambda\}$ for which there is a polynomial $p(\cdot)$ and constant c such that for each λ , f_λ can be computed by a (randomized) circuit of size $p(\lambda)$, depth $c \log(\lambda)$, and fan-in 2 using AND, OR, and NOT gates.

Definition 3.2 ($\text{AC}^0[2]$). The class of (non-uniform) $\text{AC}^0[2]$ *function families* is the set of all function families $\mathcal{F} = \{f_\lambda\}$ for which there is a polynomial $p(\cdot)$ and constant c such that for each λ , f_λ can be computed by a (randomized) circuit of size $p(\lambda)$, depth c , and unbounded fan-in using AND, OR, NOT, and PARITY gates.

Definition 3.3 ($\oplus\text{L}/\text{poly}$). $\oplus\text{L}/\text{poly}$ is the set of all boolean function families $\mathcal{F} = \{f_\lambda\}$ for which there is a constant c such that for each λ , there is a non-deterministic Turing machine $\mathcal{M}_\lambda$ such that for each input x with length λ , $\mathcal{M}_\lambda(x)$ uses at most $c \log(\lambda)$ space, and $f_\lambda(x)$ is equal to the parity of the number of accepting paths of $\mathcal{M}_\lambda(x)$.

We now recall the definitions of four sampling procedures $\{\text{LSamp}_\lambda, \text{RSamp}_\lambda, \text{ZeroSamp}_\lambda, \text{OneSamp}_\lambda\}$ in Figure 2. Note that the output of ZeroSamp_λ is always a matrix of rank $\lambda - 1$ and the output of OneSamp_λ is always a matrix of full rank [?].

LSamp_λ: For all $i, j \in [\lambda]$ and $i < j$: $r_{i,j} \xleftarrow{\\$} \{0, 1\}$ Return $\begin{pmatrix} 1 & r_{1,2} & \cdots & r_{1,\lambda-1} & r_{1,\lambda} \\ 0 & 1 & \cdots & r_{2,\lambda-1} & r_{2,\lambda} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & r_{\lambda-1,\lambda} \\ 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$	RSamp_λ: For $i = 1, \dots, \lambda - 1$ $r_i \xleftarrow{\\$} \{0, 1\}$ Return $\begin{pmatrix} 1 & 0 & \cdots & 0 & r_1 \\ 0 & 1 & \cdots & 0 & r_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & r_{\lambda-1} \\ 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$	ZeroSamp_λ: $\mathbf{R}_0 \xleftarrow{\\$} \text{LSamp}_\lambda \in \{0, 1\}^{\lambda \times \lambda}$ $\mathbf{R}_1 \xleftarrow{\\$} \text{RSamp}_\lambda \in \{0, 1\}^{\lambda \times \lambda}$ Return $\mathbf{R}_0 \mathbf{M}_0^\lambda \mathbf{R}_1 \in \{0, 1\}^{\lambda \times \lambda}$ OneSamp_λ: $\mathbf{R}_0 \xleftarrow{\\$} \text{LSamp}_\lambda$ $\mathbf{R}_1 \xleftarrow{\\$} \text{RSamp}_\lambda$ Return $\mathbf{R}_0 \mathbf{M}_1^\lambda \mathbf{R}_1 \in \{0, 1\}^{\lambda \times \lambda}$
---	---	--

Figure 2: Definitions of LSamp_λ , RSamp_λ , ZeroSamp_λ , and OneSamp_λ . By $\mathbf{M}_0^\lambda$ and $\mathbf{M}_1^\lambda$ we denote the $\lambda \times \lambda$ matrices $\mathbf{M}_0^\lambda = \begin{pmatrix} \mathbf{0} & 0 \\ \mathbf{I}_{\lambda-1} & \mathbf{0} \end{pmatrix}$ and $\mathbf{M}_1^\lambda = \begin{pmatrix} \mathbf{0} & 1 \\ \mathbf{I}_{\lambda-1} & \mathbf{0} \end{pmatrix}$ respectively.

Lemma 3.4 (Lemma 3 in [?]). *For all $\lambda \in \mathbb{N}$ and all $\mathbf{M} \in \text{ZeroSamp}_\lambda$, it holds that $\text{Ker}(\mathbf{M}) = \{\mathbf{0}, \mathbf{k}\}$ where $\mathbf{k}$ is a vector such that $\mathbf{k} \in \{0, 1\}^{\lambda-1} \times \{1\}$.*

Lemma 3.5 (Lemma 4.3 in [?]). *If $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$, for any $\{\text{adv}_\lambda\} \in \text{NC}^1$ and all sufficiently large λ , we have*

$$\begin{aligned} & |\Pr[\text{adv}_\lambda(\mathbf{M}) = 1 \mid \mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda] \\ & - \Pr[\text{adv}_\lambda(\mathbf{M}) = 1 \mid \mathbf{M} \xleftarrow{\$} \text{OneSamp}_\lambda]| \leq \text{negl}(\lambda). \end{aligned}$$

Here, we follow [?, ?, ?] to adhere to the stronger notion of $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$ mentioned in the second paragraph of Remark 3.1 in [?] for the sake of simplicity. One can easily see that if we only assume the infinitely-often version of $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$, which holds only for an infinitely large number of values of λ , then our scheme, given later, satisfies infinitely-often security.

Next, we recall a sampling algorithm RandSamp_λ that is implicitly defined in [?] and presented in Figure 3. Additionally, an inverse algorithm $\text{RandSamp}_\lambda^{-1}$ is introduced. Roughly, as proven in [?], the distribution of $(\mathbf{F} \parallel \mathbf{s}) \in \{0, 1\}^{\lambda(\lambda-1)} \times \{0, 1\}^\lambda$ generated by $\text{RandSamp}_\lambda(\mathbf{u})$ for $\mathbf{u} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}$ is the same as ZeroSamp_λ (respectively, OneSamp_λ) if $\mathbf{s}$ is in (respectively, outside) the span of $\mathbf{F}$, which happens with half probability. Hence, RandSamp_λ is indistinguishable from ZeroSamp_λ against NC^1 adversaries. One advantage of using RandSamp_λ is that it solely relies on public coins for matrix generation, and these public coins can be easily reconstructed from the matrices using $\text{RandSamp}_\lambda^{-1}$. This will help us generate the public parameter of our construction given later by making use of a URS (i.e., $\mathbf{u}$), rather than generating it by using OneSamp_λ or ZeroSamp_λ with secret randomness.

Lemma 3.6 (Lemma 6 in [?]). *If $\text{NC}^1 \subsetneq \oplus\text{L}/\text{poly}$, for any $\{\text{adv}_\lambda\} \in \text{NC}^1$, we have*

$$\begin{aligned} & |\Pr[\text{adv}_\lambda(\text{RandSamp}_\lambda(\mathbf{u})) = 1 \mid \mathbf{u} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}] \\ & - \Pr[\text{adv}_\lambda(\mathbf{M}) = 1 \mid \mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda]| \leq \text{negl}(\lambda) \end{aligned}$$

for all sufficiently large $\lambda \in \mathbb{N}$.

<u>RandSamp_λ($\mathbf{u} \in \{0, 1\}^{\lambda(\lambda+1)/2}$):</u> Parse $\mathbf{u} = (\mathbf{r}, \mathbf{s}) \in \{0, 1\}^{\lambda(\lambda-1)/2} \times \{0, 1\}^\lambda$ Parse $\mathbf{r} = (r_{i,j})_{1 \leq i < j \leq \lambda}$ and set $\mathbf{F} = \begin{pmatrix} r_{1,2} & r_{1,3} & \cdots & r_{1,\lambda-1} & r_{1,\lambda} \\ 1 & r_{2,3} & \cdots & r_{2,\lambda-1} & r_{2,\lambda} \\ 0 & 1 & \cdots & r_{3,\lambda-1} & r_{3,\lambda} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & r_{\lambda-1,\lambda} \\ 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$ Return $\mathbf{F} \parallel \mathbf{s} \in \{0, 1\}^{\lambda \times \lambda}$	<u>RandSamp_λ⁻¹($\mathbf{M} \in \{0, 1\}^{\lambda \times \lambda}$):</u> Parse $\mathbf{M} = \mathbf{F} \parallel \mathbf{s}$ where $\mathbf{F} = \begin{pmatrix} r_{1,2} & r_{1,3} & \cdots & r_{1,\lambda-1} & r_{1,\lambda} \\ 1 & r_{2,3} & \cdots & r_{2,\lambda-1} & r_{2,\lambda} \\ 0 & 1 & \cdots & r_{3,\lambda-1} & r_{3,\lambda} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & r_{\lambda-1,\lambda} \\ 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$ Abort if $\mathbf{M}$ does not conform to the form Set $\mathbf{r} = (r_{i,j})_{1 \leq i < j \leq \lambda}$ Return $\mathbf{u} = (\mathbf{r}, \mathbf{s}) \in \{0, 1\}^{\lambda(\lambda+1)/2}$
---	--

Figure 3: The definition of RandSamp_λ and RandSamp_λ⁻¹.

<u>SymR_λ:</u> Output a matrix $\mathbf{R} \in \{0, 1\}^{\lambda \times \lambda}$ given by $\mathbf{R}_{ij} = \begin{cases} \text{a random bit, if } i \leq j \\ \mathbf{R}_{ji}, \text{ if } i > j \end{cases}$

Figure 4: The definition of SymR_λ.

3.2 Our Construction

In this subsection, we give our construction of multi-party NIKE.

Core lemma. Before giving our construction, we define an algorithm SymR_λ outputting uniformly random symmetric matrices, and then propose a core lemma upon which we will heavily rely later.

Lemma 3.7 *If $\text{NC}^1 \subsetneq \oplus \text{L}/\text{poly}$ holds, then for any $\{\text{adv}_\lambda\} \in \text{NC}^1$, any sufficiently large $\lambda \in \mathbb{N}$, and any polynomial $n = n(\lambda)$ in λ , we have:*

$$|\Pr[\text{adv}_\lambda(\mathbf{M}, (\widetilde{\mathbf{M}}\mathbf{R}_i)_{i \in [n]}) = 1] - \Pr[\text{adv}_\lambda(\mathbf{M}, (\widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda)_{i \in [n]}) = 1]| \leq \text{negl}(\lambda)$$

where $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$, $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$ and $\mathbf{R}_1, \dots, \mathbf{R}_n \xleftarrow{\$} \text{SymR}_\lambda$.

Proof. Let $\lambda \in \mathbb{N}$ be any sufficiently large security parameter. Let $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$, $\mathbf{M}' \xleftarrow{\$} \text{OneSamp}_\lambda$, $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$, and $\widetilde{\mathbf{M}}' = \mathbf{M}'^\top \mathbf{M}'$. Since the multiplication of two matrices can be performed in NC^1 , according to Theorem 3.5, we immediately have $|\Pr[\text{adv}_\lambda(\mathbf{M}, \widetilde{\mathbf{M}}) = 1] - \Pr[\text{adv}_\lambda(\mathbf{M}', \widetilde{\mathbf{M}}') = 1]| \leq \text{negl}(\lambda)$, i.e.,

$$|\Pr[\text{adv}_\lambda(\mathbf{M}, (\widetilde{\mathbf{M}}\mathbf{R}_i)_{i \in [n]}) = 1] - \Pr[\text{adv}_\lambda(\mathbf{M}', (\widetilde{\mathbf{M}}'\mathbf{R}_i)_{i \in [n]}) = 1]| \leq \text{negl}(\lambda), \quad (1)$$

$$|\Pr[\text{adv}_\lambda(\mathbf{M}', (\widetilde{\mathbf{M}}'\mathbf{R}_i + \mathbf{I}_\lambda)_{i \in [n]}) = 1] - \Pr[\text{adv}_\lambda(\mathbf{M}, (\widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda)_{i \in [n]}) = 1]| \leq \text{negl}(\lambda), \quad (2)$$

where $\mathbf{R}_1, \dots, \mathbf{R}_n \xleftarrow{\$} \text{SymR}_\lambda$.

Recall that any $\mathbf{M}' \in \text{OneSamp}_\lambda$ is of full rank and has an inverse matrix $\mathbf{M}'^{-1}$. Hence, $\widetilde{\mathbf{M}}' = \mathbf{M}'^\top \mathbf{M}'$ also has an inverse matrix $\widetilde{\mathbf{M}}'^{-1} = \mathbf{M}'^{-1} \mathbf{M}'^{-1\top}$ and is of full rank. Furthermore, since $\widetilde{\mathbf{M}}'^{-1}$ is a symmetric matrix, for all $i \in [n]$, the following distributions are identical:

$$(\mathbf{R}_i + \widetilde{\mathbf{M}}'^{-1})_{i \in [n]} \text{ and } (\mathbf{R}_i)_{i \in [n]},$$

where $\mathbf{R}_1, \dots, \mathbf{R}_n \xleftarrow{\$} \text{SymR}_\lambda$. Hence, the following two distributions are also identical:

$$(\mathbf{M}', (\widetilde{\mathbf{M}}'\mathbf{R}_i)_{i \in [n]}) \text{ and } (\mathbf{M}', (\widetilde{\mathbf{M}}'(\mathbf{R}_i + (\widetilde{\mathbf{M}}')^{-1}))_{i \in [n]}) = (\widetilde{\mathbf{M}}'\mathbf{R}_i + \mathbf{I})_{i \in [n]}).$$

Combining this with Equations (1) and (2), Theorem 3.7 immediately follows. $\square$

$\text{Gen}_\lambda(\text{urs} \in \{0, 1\}^{\lambda(\lambda+1)/2}):$ $\mathbf{M} = \text{RandSamp}_\lambda(\text{urs})$ $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$ $\text{sk} = \mathbf{R} \xleftarrow{\$} \text{SymR}_\lambda, \text{pk} = \widetilde{\mathbf{M}}\mathbf{R}$ Return (pk, sk)	$\text{Share}_\lambda(i, \text{sk}_i, (\text{pk}_j)_{j \in [n_p]}):$ Parse $(\text{pk}_j)_{j \in [n_p]} = (\mathbf{P}_j)_{j \in [n_p]}$ and $\text{sk}_i = \mathbf{R}_i$ $\mathbf{K} = (\prod_{j < i} \mathbf{P}_j^\top) \mathbf{R}_i (\prod_{j > i} \mathbf{P}_j)$ Let key be the bottom-right bit of $\mathbf{K}$ return key
---	--

Figure 5: Definition of NCNIKE = $\{\text{Gen}_\lambda, \text{Share}_\lambda\}$.

Construction. We now give our construction of multi-party NIKE NCNIKE in the bounded-parallel-time model in Figure 5.

Theorem 3.8 *If $\text{NC}^1 \not\subseteq \oplus\text{L}/\text{poly}$ holds, then for any constant n_p , NCNIKE is an $\text{AC}^0[2]$ - n_p -NIKE with 0-correctness (i.e., perfect correctness) and $\text{NC}^1\text{-negl}(\lambda)$ -security.*

Proof. Complexity. First we note that $\{\text{Gen}_\lambda, \text{Share}_\lambda\}$ are computable in $\text{AC}^0[2]$ since they only involve operations including multiplication of a constant number of matrices and sampling random bits. **Correctness.** Let $(\mathbf{P}_i = \mathbf{M}\mathbf{R}_i, \mathbf{R}_i)_{i \in [n_p]}$ be n_p public/secret key pairs generated by Gen_λ . For all $i \in [n_p]$, we have $\mathbf{P}_i^\top = (\widetilde{\mathbf{M}}\mathbf{R}_i)^\top = \mathbf{R}_i^\top \widetilde{\mathbf{M}}^\top = \mathbf{R}_i^\top \widetilde{\mathbf{M}}$. Moreover, for any $i \in [n_p]$, we have

$$\prod_{j < i} \mathbf{P}_j^\top \mathbf{R}_i \prod_{j > i} \mathbf{P}_j = \prod_{j < i} (\mathbf{R}_j \widetilde{\mathbf{M}}) \mathbf{R}_i \prod_{j > i} \mathbf{P}_j = \mathbf{R}_1 \prod_{1 < j \leq i} (\widetilde{\mathbf{M}}\mathbf{R}_j) \prod_{j > i} \mathbf{P}_j = \mathbf{R}_1 \prod_{j > 1} \mathbf{P}_j.$$

Therefore, for all $i, i' \in [n_p]$, we have

$$\text{Share}_\lambda(i, \text{sk}_i, (\text{pk}_j)_{j \in [n_p]}) = \text{Share}_\lambda(j, \text{sk}_j, (\text{pk}_i)_{i \in [n_p]}) = \text{key},$$

where key is the bottom-right bit of $\mathbf{R}_1 \prod_{j > 1} \mathbf{P}_j$, completing the proof of correctness.

Security. Let λ be the security parameter and $\mathcal{A} = \{\text{adv}_\lambda\} \in \text{NC}^1$ be any adversary against the security of NCNIKE. We prove the security of NCNIKE via a sequence of hybrid games as in Figure 6.

Games $G_0, \boxed{G_1}, \boxed{G_2}, \boxed{G_3}, \boxed{G_4}$
$\mathbf{M} = \text{RandSamp}_\lambda(\text{urs})$ where $\text{urs} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}$ $\mathbf{M} \xleftarrow{\\$} \text{ZeroSamp}_\lambda$, $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$ $\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$ for $i = 1, \dots, n_p$ $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i$ for $i = 1, \dots, n_p$ $\mathbf{P}_1 = \widetilde{\mathbf{M}}\mathbf{R}_1, \mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda$ for $i = 2, \dots, n_p$ $\mathbf{K} = \mathbf{R}_1 \mathbf{P}_2 \cdots \mathbf{P}_{n_p}$ $\mathbf{K} = (\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top) \mathbf{P}_2 \cdots \mathbf{P}_{n_p}$ Set key as the bottom-right bit of $\mathbf{K}$ $\text{key} \xleftarrow{\\$} \{0, 1\}$ Return $(\text{urs}, (\text{pk}_i = \mathbf{P}_i)_{i \in [n_p]}, \text{key})$ to adv_λ Output whatever adv_λ outputs

Figure 6: The challengers in G_0, G_1, G_2, G_3 , and G_4 for the proof of Theorem 3.8. $\mathbf{m}$ is the non-zero vector in $\text{Ker}(\mathbf{M})$ guaranteed by Theorem 3.4.

G_0 . This is the original security game where adv_λ receives $\text{urs} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}$, $(\text{pk}_i)_{i \in [n_p]}$ where $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [n_p]$, and $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$, and outputs a bit β .

G_1 . G_1 and G_0 only differ in the way that $\mathbf{M}$ is generated, namely, $\mathbf{M}$ is generated by ZeroSamp_λ rather than RandSamp_λ in G_1 .

Lemma 3.9 $|\Pr[G_1^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[G_0^{\text{adv}_\lambda} \Rightarrow 1]| \leq \text{negl}(\lambda)$.

Proof. We construct an adversary $\mathcal{B}^0 = \{\text{adv}_\lambda^0\} \in \text{NC}^1$ in the game described in Theorem 3.6 as follows.

On receiving $\mathbf{M}$ sampled as $\mathbf{M} = \text{RandSamp}_\lambda(\text{urs})$ where $\text{urs} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}$ or $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$, adv_λ^0 sets $\text{urs} = \text{RandSamp}_\lambda^{-1}(\mathbf{M})$, runs $\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$ and $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i$ for all $i = 1, \dots, n_p$, computes $\mathbf{K} = \mathbf{R}_1 \prod_{i>1} \mathbf{P}_i$, and sets key as the bottom-right bit of $\mathbf{K}$. Next it sends $(\text{urs}, (\mathbf{P}_i)_{i \in [n_p]}, \text{key})$ to adv_λ . When adv_λ returns $\beta \in \{0, 1\}$, adv_λ^0 returns β as well.

First we note that all operations in adv_λ^0 are NC^1 , since adv_λ^0 only samples random bits, computes multiplication of a constant number of matrices, and runs adv_λ . Moreover, when $\mathbf{M}$ is generated as $\mathbf{M} = \text{RandSamp}_\lambda(\text{urs})$ where $\text{urs} \xleftarrow{\$} \{0, 1\}^{\lambda(\lambda+1)/2}$ (respectively, $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$), the view of adv_λ is identical to its view in $\mathbf{G}_0$ (respectively, $\mathbf{G}_1$). Hence, the advantage of adv_λ^0 is

$$|\Pr[\mathbf{G}_1^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\text{adv}_\lambda} \Rightarrow 1]|,$$

which is negligible according to Theorem 3.6, completing this part of the proof. $\square$

$\mathbf{G}_2$. $\mathbf{G}_2$ and $\mathbf{G}_1$ only differ in the way that $\mathbf{P}_2, \dots, \mathbf{P}_{n_p}$ are generated, namely, $\mathbf{P}_i$ is generated as $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda$ where $\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$ for $i = 2, \dots, n_p$ in $\mathbf{G}_2$.

Lemma 3.10 $|\Pr[\mathbf{G}_2^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\text{adv}_\lambda} \Rightarrow 1]| \leq \text{negl}(\lambda)$.

Proof. We construct an adversary $\mathcal{B}^1 = \{\text{adv}_\lambda^1\} \in \text{NC}^1$ in the game described in Theorem 3.7 as follows.

On receiving $\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda$ and $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i$ or $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda$, where $\widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$ and $\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$, for $i = 2, \dots, n_p$, adv_λ^1 sets $\text{urs} = \text{RandSamp}_\lambda^{-1}(\mathbf{M})$, samples $\mathbf{R}_1 \xleftarrow{\$} \text{SymR}_\lambda$, sets $\mathbf{P}_1 = \widetilde{\mathbf{M}}\mathbf{R}_1$, computes $\mathbf{K} = \mathbf{R}_1 \prod_{i=2}^{n_p} \mathbf{P}_i$, and sets key as the bottom-right bit of $\mathbf{K}$. Next it sends $(\text{urs}, (\mathbf{P}_i)_{i \in [n_p]}, \text{key})$ to adv_λ . When adv_λ returns $\beta \in \{0, 1\}$, adv_λ^1 returns β as well.

First we note that all operations in adv_λ^1 are NC^1 , since adv_λ^1 only samples random bits, computes multiplication of a constant number of matrices, and runs adv_λ . Moreover, when $\mathbf{P}_i$ is generated as $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i$ (respectively, $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda$) for all $i \in [n_p] \setminus \{1\}$, the view of adv_λ is identical to its view in $\mathbf{G}_1$ (respectively, $\mathbf{G}_2$). Hence, the advantage of adv_λ^1 is

$$|\Pr[\mathbf{G}_2^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\text{adv}_\lambda} \Rightarrow 1]|,$$

which is negligible according to Theorem 3.7, completing this part of the proof. $\square$

$\mathbf{G}_3$. $\mathbf{G}_3$ is exactly the same as $\mathbf{G}_2$ except that when generating $\mathbf{K}$, we replace $\mathbf{R}_1$ by $(\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top)$. Here, $\mu \xleftarrow{\$} \{0, 1\}$ and $\mathbf{m} \in \{0, 1\}^{\lambda-1} \times \{1\}$ is the non-zero vector in $\text{Ker}(\mathbf{M})$, which must exist according to Theorem 3.4.

Lemma 3.11 $\Pr[\mathbf{G}_3^{\text{adv}_\lambda} \Rightarrow 1] = \Pr[\mathbf{G}_2^{\text{adv}_\lambda} \Rightarrow 1]$.

Proof. First we note that the distribution of $\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top$ is uniform in SymR_λ for $\mathbf{R}_1 \xleftarrow{\$} \text{SymR}_\lambda$, since $\mathbf{m} \cdot \mu \cdot \mathbf{m}^\top$ is a symmetric matrix. Moreover, we have $\mathbf{P}_1 = \widetilde{\mathbf{M}}(\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top) = \widetilde{\mathbf{M}}\mathbf{R}_1$. Then Theorem 3.11 immediately follows from the fact that the view of adv_λ in $\mathbf{G}_3$ is identical to its view in $\mathbf{G}_2$. $\square$

$\mathbf{G}_4$. $\mathbf{G}_4$ is exactly the same as $\mathbf{G}_3$ except that the challenger sets the session key key as a random bit.

Lemma 3.12 $\Pr[\mathbf{G}_4^{\text{adv}_\lambda} \Rightarrow 1] = \Pr[\mathbf{G}_3^{\text{adv}_\lambda} \Rightarrow 1]$.

Proof. First we note that $\mathbf{P}_2 = \widetilde{\mathbf{M}}\mathbf{R}_2 + \mathbf{I}_\lambda$ in $\mathbf{G}_3$ and $\mathbf{G}_4$. Assuming that $\prod_{j=2}^i \mathbf{P}_j = \widetilde{\mathbf{M}}\mathbf{S}_i + \mathbf{I}_\lambda$ for some $2 \leq i < n_p$ and some $\mathbf{S}_i \in \{0, 1\}^{\lambda \times \lambda}$, we have

$$\begin{aligned} \prod_{j=2}^{i+1} \mathbf{P}_j &= (\widetilde{\mathbf{M}}\mathbf{S}_i + \mathbf{I}_\lambda)\mathbf{P}_{i+1} = \widetilde{\mathbf{M}}\mathbf{S}_i\mathbf{P}_{i+1} + \mathbf{P}_{i+1} \\ &= \widetilde{\mathbf{M}}\mathbf{S}_i\mathbf{P}_{i+1} + \widetilde{\mathbf{M}}\mathbf{R}_{i+1} + \mathbf{I}_\lambda = \widetilde{\mathbf{M}}(\mathbf{S}_i\mathbf{P}_{i+1} + \mathbf{R}_{i+1}) + \mathbf{I}_\lambda, \end{aligned}$$

namely, we have $\prod_{j=2}^{i+1} \mathbf{P}_j = \widetilde{\mathbf{M}}\mathbf{S}_{i+1} + \mathbf{I}_\lambda$ for $\mathbf{S}_{i+1} = \mathbf{S}_i\mathbf{P}_{i+1} + \mathbf{R}_{i+1}$.

Then by mathematical induction, we have $\prod_{j=2}^{n_p} \mathbf{P}_j = \widetilde{\mathbf{M}}\mathbf{S}_{n_p} + \mathbf{I}_\lambda$ for some $\mathbf{S}_{n_p} \in \{0,1\}^{\lambda \times \lambda}$, i.e., the rightmost column vector in $\prod_{j=2}^{n_p} \mathbf{P}_j$ is in the form of $\widetilde{\mathbf{M}}\mathbf{s}_k + \mathbf{e}_\lambda$ for some $\mathbf{s}_k \in \{0,1\}^\lambda$ and $\mathbf{e}_\lambda = (0, \dots, 0, 1)^\top$. Moreover, since the last bit in $\mathbf{m}$ is 1 according to Theorem 3.4, the bottom vector of $\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top$ must be in the form of $\mathbf{r}_1^\top + \mu \cdot \mathbf{m}^\top$, where $\mathbf{r}_1^\top$ denotes the bottom vector in $\mathbf{R}_1$. Let key be the bottom-right bit of $\mathbf{K}$ in $\mathbf{G}_2$. Since $\mathbf{m} \in \text{Ker}(\mathbf{M})$, we have $\mathbf{m}^\top \widetilde{\mathbf{M}} = (\mathbf{M}\mathbf{m})^\top \mathbf{M} = \mathbf{0}^\top$. We have

$$\text{key} = (\mathbf{r}_1^\top + \mu \cdot \mathbf{m}^\top)(\widetilde{\mathbf{M}}\mathbf{s}_k + \mathbf{e}_\lambda) = \mathbf{r}_1^\top(\widetilde{\mathbf{M}}\mathbf{s}_k + \mathbf{e}_\lambda) + \underbrace{\mu \cdot \mathbf{m}^\top \widetilde{\mathbf{M}}\mathbf{s}_k}_{=0} + \underbrace{\mu \cdot \mathbf{m}^\top \mathbf{e}_\lambda}_{=1}.$$

Since μ is a random bit and adv_λ obtains no information on μ except for key , key is a random bit in the view of adv_λ as well, completing this part of the proof. $\square$

Games $H_0, \boxed{H_1}, \boxed{H_2}, \boxed{H_3}$

$\mathbf{M} \xleftarrow{\$} \text{ZeroSamp}_\lambda \quad \boxed{\mathbf{M} = \text{RandSamp}_\lambda(\text{urs}) \text{ where } \text{urs} \xleftarrow{\$} \{0,1\}^{\lambda(\lambda+1)/2}}, \widetilde{\mathbf{M}} = \mathbf{M}^\top \mathbf{M}$

$\mathbf{R}_i \xleftarrow{\$} \text{SymR}_\lambda$ for $i = 1, \dots, n_p$

$\mathbf{P}_1 = \widetilde{\mathbf{M}}\mathbf{R}_1, \mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i + \mathbf{I}_\lambda$ for $i = 2, \dots, n_p$ $\mathbf{P}_i = \widetilde{\mathbf{M}}\mathbf{R}_i$ for $i = 2, \dots, n_p$

$\mathbf{K} = (\mathbf{R}_1 + \mathbf{m} \cdot \mu \cdot \mathbf{m}^\top)\mathbf{P}_2 \cdots \mathbf{P}_{n_p}$ $\mathbf{K} = \mathbf{R}_1\mathbf{P}_2 \cdots \mathbf{P}_{n_p}$

Set $\text{key} \xleftarrow{\$} \{0,1\}$

Return $(\text{urs}, (\mathbf{p}_{\mathbf{k}_i} = \mathbf{P}_i)_{i \in [n_p]}, \text{key})$ to adv_λ

Output whatever adv_λ outputs

Figure 7: The challengers in H_0, H_1, H_2 , and H_3 for the proof of Theorem 3.8. $\mathbf{m}$ is the non-zero vector in $\text{Ker}(\mathbf{M})$ guaranteed by Theorem 3.4.

We now do all the previous steps in the reverse order as in Figure 7. Note that the view of the adversary in H_0 (respectively, H_3) is identical to its view in G_4 (respectively, the honest game where key is a random bit). By using the above arguments in a reverse order, we have the following lemma.

Lemma 3.13 $|\Pr[H_3^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[H_0^{\text{adv}_\lambda} \Rightarrow 1]| \leq \text{negl}(\lambda)$.

Putting all the above together, Theorem 3.8 immediately follows. $\square$

Extension to IB-NIKE. We can also show that our construction satisfies 0-restricted extendability (see Theorem 2.3), and thus can be converted into a restricted BPRF, which in turn implies a multi-party IB-NIKE in the bounded-parallel-time model. A similar argument can also be made for our constructions in the bounded-time model and the BSM, while those constructions further satisfy (full) extendability (see Theorem 2.2) and can be converted into full-fledged BPRFs. We refer the reader to Section A for the full details.

4 Fine-Grained Multi-Party NIKE based on Zero- k -Clique

In this section, we propose a fine-grained multi-party NIKE based on Zero- k -Clique. All algorithms and adversaries in this section are modeled as word-RAMs with $O(\log \lambda)$ -bit words. The resulting scheme is computable in $\tilde{O}(\lambda^{n_p+k-1})$ and secure against adversaries running in time $\tilde{O}(\lambda^{n_p+k-\Omega(1)})$ for any constant number n_p of parties.

4.1 Zero- k -Clique Problem and Its Properties

We now introduce the Zero- k -Clique problem, which is the set of k -partite graphs with weighted edges. Each graph is said to have a solution if there is a k -clique (i.e., a complete subgraph) where the sum of edge weights within the clique is 0. Below, for any two nodes v and u , $\text{weight}(v, u)$ denotes the weight of the edge (v, u) .

Definition 4.1 (Zero- k -Clique- R problem). Let $\lambda \in \mathbb{N}$ be the security parameter. For $k > 2$, a *Zero- k -Clique- R problem* $P_{k,R}$ is the set of all k -partite graphs with λ nodes in each partition and the weight of each edge being in $\mathbb{Z}_R$. Each graph with k partitions $\mathcal{P}_1, \dots, \mathcal{P}_k$ is required to be complete, namely, there is an edge between two nodes $v \in \mathcal{P}_i$ and $u \in \mathcal{P}_j$ if and only if $i \neq j$. $(v_1, \dots, v_k) \in \mathcal{P}_1 \times \dots \times \mathcal{P}_k$ is said to be a solution of I if for each $v_i, v_j \wedge i \neq j$, we have $\sum_{i \in [k]} \sum_{j \in [k] \wedge i \neq j} \text{weight}(v_i, v_j) \equiv 0 \pmod R$.

For any $P_{k,R}$, we define two distributions $\mathcal{D}_0[R]$ and $\mathcal{D}_1[R]$. $\mathcal{D}_0[R]$ (respectively, $\mathcal{D}_1[R]$) is the uniform distribution over the instances in $P_{k,R}$ having no solutions (respectively, having a single solution).

The Zero- k -Clique- R hypothesis says that it is easy to verify a solution of a random instance in the Zero- k -Clique problem and difficult to find a solution given a random instance.

Definition 4.2 (Strong Zero- k -Clique- R hypothesis and range $R = \lambda^{ck}$). For any constant $c > 1$, $\lambda \in \mathbb{N}$, and $R = \lambda^{ck}$, the *strong Zero- k -Clique- R hypothesis* holds if

- there exists an algorithm **Verify** with running time $O(\lambda^{k(1-\delta)})$ for some constant $\delta > 0$ such that $\text{Verify}(I, \mathbf{w}) = 1$ if I has a solution for an arbitrary witness $\mathbf{w}$ of an instance I with one solution,
- for any algorithm $\mathcal{A}$ with running time $O(\lambda^{k(1-\delta)})$ for some constant $\delta > 0$, $\Pr[\text{adv}_\lambda(I) = \mathbf{w}] \leq 1/100$ where $I \leftarrow \mathcal{D}_1[R]$ and $\mathbf{w}$ is an arbitrary witness of the instance I with one solution.

Remark. Below when mentioning the strong Zero- k -Clique hypothesis, we are always referring to the strong Zero- k -Clique hypothesis with $k > 2$ and $R = \lambda^{ck}$.

Theorem 4.3 (Theorem 15 in [?]). *If the strong Zero- k -Clique- R hypothesis holds, then for every probabilistic word-RAM family $\{\text{adv}_\lambda\}$ with running time $O((\ell(\lambda)T(\lambda))^{1-\delta})$ for some constant $\delta > 0$, we have*

$$\Pr[\text{adv}_\lambda((I_j)_{j=1}^{\ell(\lambda)}) = i] \leq \epsilon,$$

where $I_i \leftarrow \mathcal{D}_1[R]$ for $i \leftarrow [\ell(\lambda)]$, $I_j \leftarrow \mathcal{D}_0[R]$ for all $j \in [\ell(\lambda)] \setminus \{i\}$, $\ell(\lambda) = \lambda^{\Omega(1)}$, $\epsilon \leq 1/200$, and $T(\lambda) = \lambda^k$.

Next we recall a word-RAM family $\{\text{Generate}_\lambda\}$ such that $\text{Generate}_\lambda(R, k, b)$ produces an instance of the Zero- k -Clique- R problem drawn from a distribution within total variation distance at most $2\lambda^k/R$ of $\mathcal{D}_b[R]$ defined in [?] as in Figure 8.

Generate $_\lambda(R, k, b)$:

Generate $I \in P_{k,R}$ with partitions $\mathcal{P}_1, \dots, \mathcal{P}_k$ such that $\text{weight}(v_i, v_j) \leftarrow \mathbb{Z}_R$ for all $i \neq j$, all $v_i \in \mathcal{P}_i$, and all $v_j \in \mathcal{P}_j$

If $b = 0$, return I

Otherwise, sample $i, j \leftarrow [k] \wedge i \neq j$, $v_1 \leftarrow \mathcal{P}_1, \dots, v_k \leftarrow \mathcal{P}_k$ and set $\text{weight}(v_i, v_j) = -\sum_{(i', j') \neq (i, j)} \text{weight}(v_{i'}, v_{j'})$

Return I

Figure 8: Definition of Generate_λ [?].

Theorem 4.4 (Theorem 14 in [?]). *If the strong Zero- k -Clique- R hypothesis holds, then we have $\Delta(\text{Generate}_\lambda(R, k, b), \mathcal{D}_b[R]) \leq 2\lambda^k/R$, for all $b \in \{0, 1\}$, where Generate_λ runs in time $O(\lambda^2)$.*

The following corollary follows immediately from Theorem 4.4.

Corollary 4.5 *If the strong Zero- k -Clique- R hypothesis holds, we have*

$$\Delta((I_i^j)_{j \in [\ell], i \in [\mathbf{n}_p]}, (\mathcal{D}_{b_i^j}[R])_{j \in [\ell], i \in [\mathbf{n}_p]}) \leq \sum_{j \in [\ell], i \in [\mathbf{n}_p]} \Delta(I_i^j, \mathcal{D}_{b_i^j}[R]) = 2\ell \mathbf{n}_p \lambda^k / R,$$

where $I_i^j \leftarrow \text{Generate}_\lambda(R, k, b_i^j)$, $b_i^j \in \{0, 1\}$ for all $i \in [\mathbf{n}_p]$ and $j \in [\ell]$, $\ell = \lambda^{\Omega(1)}$, and any constant $\mathbf{n}_p$.

We now recall a theorem given in [?]. Roughly, it says that there exists a word-RAM family $\{\text{Split}_\lambda\}$ such that for an instance I of the Zero- k -Clique- R problem with one solution, $\text{Split}_\lambda(I)$ outputs two slightly smaller instances that both have solutions in $O(\lambda^{k(1-\delta)})$ time for some constant $\delta > 0$.

Theorem 4.6 (Theorem 16 in [?]). *If the strong Zero- k -Clique- R hypothesis holds, then there exists a word-RAM family $\{\text{Split}_\lambda\}$ with running time $O(\lambda^{k(1-\delta)})$ for some constant $\delta > 0$ such that*

- $\Delta((\hat{I}_1^i, \hat{I}_2^i), (\mathcal{D}_0[\sqrt{R}] \times \mathcal{D}_0[\sqrt{R}])) \leq \binom{k}{2} \cdot 4^{\binom{k}{2}} \cdot 3\lambda^k / \sqrt{R}$ for all $i \in [\binom{k}{2}]$ where $I \stackrel{s}{\leftarrow} \mathcal{D}_0[R]$, $(\hat{I}_1^j, \hat{I}_2^j)_{j \in [\binom{k}{2}]} \stackrel{s}{\leftarrow} \text{Split}_\lambda(I)$.
- $\Delta((\hat{I}_1^i, \hat{I}_2^i), (\mathcal{D}_1[\sqrt{R}] \times \mathcal{D}_1[\sqrt{R}])) \leq \binom{k}{2} \cdot 4^{\binom{k}{2}} \cdot 3\lambda^k / \sqrt{R}$ for some $i \in [\binom{k}{2}]$ where $I \stackrel{s}{\leftarrow} \mathcal{D}_1[R]$, $(\hat{I}_1^j, \hat{I}_2^j)_{j \in [\binom{k}{2}]} \stackrel{s}{\leftarrow} \text{Split}_\lambda(I)$.

4.2 Construction

Generalized splitting algorithm. Before giving our construction, we propose generalized splitting algorithms $\{\mathbf{n}_p\text{-Split}_\lambda\}$. For any constant $\mathbf{n}_p$, let

$$L = \lceil \log_2 \mathbf{n}_p \rceil \quad \text{and} \quad \bar{\mathbf{n}}_p = 2^L.$$

The binary recursive splitter is run to depth L , producing $\bar{\mathbf{n}}_p$ output instances in each list; $\mathbf{n}_p\text{-Split}_\lambda$ returns the first $\mathbf{n}_p$ of them. Since discarding output coordinates cannot increase total variation distance, the resulting $\mathbf{n}_p$ -tuple inherits the bound for the $\bar{\mathbf{n}}_p$ -tuple. The algorithm generates $m = \binom{k}{2}^L$ lists of instances in time $O(\lambda^{k(1-\delta)})$ for some constant $\delta > 0$, where the number of solutions for each instance remains unchanged.

$\mathbf{n}_p\text{-Split}_\lambda(I, L)$:
 Return $\text{Split}_\lambda(I)$ if $L = 1$
 Set $L = L - 1$ and run $(I_1^t, \dots, I_{2^L}^t)_{t \in [m]} \stackrel{s}{\leftarrow} \mathbf{n}_p\text{-Split}_\lambda(I, L)$
 For each $t \in [m]$ and each $i \in [2^L]$, run $(\hat{I}_{2i-1}^{j \times t}, \hat{I}_{2i}^{j \times t})_{j \in [\binom{k}{2}]} \stackrel{s}{\leftarrow} \text{Split}_\lambda(I_i^t)$
 Return $(\hat{I}_1^j, \dots, \hat{I}_{2^{L+1}}^j)_{j \in [m \cdot \binom{k}{2}]}$

Figure 9: The construction of $\mathbf{n}_p\text{-Split}_\lambda$. By m we denote the number of lists, which is initialized with $\binom{k}{2}$. Let $\{\text{Split}_\lambda\}$ be the word-RAM family from Theorem 4.6.

When the outermost call uses $L = \lceil \log_2 \mathbf{n}_p \rceil$, each returned tuple is truncated to its first $\mathbf{n}_p$ coordinates.

Theorem 4.7 *If the strong Zero- k -Clique- R hypothesis holds, then, for $L = \lceil \log_2 \mathbf{n}_p \rceil$, $\bar{\mathbf{n}}_p = 2^L$, and $m = \binom{k}{2}^L$, we have*

$$\Delta((\hat{I}_1^i, \dots, \hat{I}_{\bar{\mathbf{n}}_p}^i), \mathcal{D}_0[\sqrt[\bar{\mathbf{n}}_p]{R}]) \leq (2\bar{\mathbf{n}}_p - 1) \cdot \binom{k}{2} \cdot 4^{\binom{k}{2}} \cdot 3\lambda^k / \sqrt[\bar{\mathbf{n}}_p]{R}$$

for all $i \in [m]$ where $(\hat{I}_1^j, \dots, \hat{I}_{\bar{\mathbf{n}}_p}^j)_{j \in [m]} \stackrel{s}{\leftarrow} \mathbf{n}_p\text{-Split}_\lambda(I)$ for $I \stackrel{s}{\leftarrow} \mathcal{D}_0[R]$, and

$$\Delta((\hat{I}_1^i, \dots, \hat{I}_{\bar{\mathbf{n}}_p}^i), \mathcal{D}_1[\sqrt[\bar{\mathbf{n}}_p]{R}]) \leq (2\bar{\mathbf{n}}_p - 1) \cdot \binom{k}{2} \cdot 4^{\binom{k}{2}} \cdot 3\lambda^k / \sqrt[\bar{\mathbf{n}}_p]{R}$$

for some $i \in [m]$ where $(\hat{I}_1^j, \dots, \hat{I}_{\bar{\mathbf{n}}_p}^j)_{j \in [m]} \stackrel{s}{\leftarrow} \mathbf{n}_p\text{-Split}_\lambda(I)$ for $I \stackrel{s}{\leftarrow} \mathcal{D}_1[R]$.

Proof. We first note that $\mathbf{n}_p\text{-Split}_\lambda$ runs Split_λ only a constant number of times since $\mathbf{n}_p$ is constant. Hence its running time is asymptotically the same as that of Split_λ , namely $O(\lambda^{k(1-\delta)})$.

Let $L = \lceil \log_2 \mathbf{n}_p \rceil$ and $\bar{\mathbf{n}}_p = 2^L$. Consider first the case $I \stackrel{s}{\leftarrow} \mathcal{D}_0[R]$. After r levels of the binary splitting recursion, each output list contains 2^r instances. By repeatedly applying Theorem 4.6 and taking a union bound over the 2^r split calls used to pass from level r to level $r+1$, for every output list after L levels we have

$$\Delta((\hat{I}_1, \dots, \hat{I}_{\bar{\mathbf{n}}_p}), \mathcal{D}_0[\sqrt[\bar{\mathbf{n}}_p]{R}]) \leq \sum_{r=0}^{L-1} 2^r \binom{k}{2} 4^{\binom{k}{2}} 3\lambda^k / 2^{r+1} \sqrt{R}.$$

Since $2^{r+1} \leq \bar{n}_p$ for every $r \leq L-1$, we have $1/2^{r+1}\sqrt{R} \leq 1/\sqrt[r]{R}$. Therefore,

$$\Delta\left((\hat{I}_1, \dots, \hat{I}_{\bar{n}_p}), \mathcal{D}_0[\sqrt[r]{R}]^{\bar{n}_p}\right) \leq (2\bar{n}_p - 1) \binom{k}{2} 4^{\binom{k}{2}} 3\lambda^k / \sqrt[r]{R}.$$

The actual $\mathbf{n}_p\text{-Split}_\lambda$ output discards the final $\bar{n}_p - n_p$ coordinates. By the data-processing property of total variation distance, this truncation cannot increase the distance. This gives the claimed zero-solution bound.

The one-solution case is identical except that at each level we follow one good child guaranteed by Theorem 4.6. Thus there exists one output list whose full $\bar{n}_p$ -tuple is within the same distance of $\mathcal{D}_1[\sqrt[r]{R}]^{\bar{n}_p}$. Truncating this list to its first n_p coordinates again cannot increase the distance, which proves the claimed one-solution bound. $\square$

Our construction. We now present our construction of fine-grained multi-party NIKE TNIKE based on the Zero- k -Clique hypothesis in Figure 10.

$\text{Gen}_\lambda(\perp)$:
 Sample a random subset $\mathcal{S} \subset [\ell]$ such that $|\mathcal{S}| = q$
 For all $j \in [\ell]$
 $I^j = \text{Generate}_\lambda(R, k, 1)$ if $j \in \mathcal{S}$ and $I^j = \text{Generate}_\lambda(R, k, 0)$ otherwise
 Return $(\mathbf{pk} = (I^j)_{j \in [\ell]}, \mathbf{sk} = \mathcal{S})$

$\text{Share}_\lambda(i, \mathbf{sk}_i, (\mathbf{pk}_j)_{j \in [n_p]})$:
 Parse $(\mathbf{pk}_t = (I_t^j)_{j \in [\ell]})_{t \in [n_p]}, \mathbf{sk}_i = \mathcal{S}_i$.
 For $t \in [n_p] \setminus \{i\}, \mathcal{T} = \emptyset$
 For every $s \in \mathcal{S}_i$, check by brute forcing whether I_t^s has one solution
 If the check passes, $\mathcal{T} = \mathcal{T} \cup \{s\}$
 $\mathcal{S}_i = \mathcal{S}_i \cap \mathcal{T}$
 Return $\mathbf{key} = \bigoplus_{j=1}^{|\mathcal{S}_i|} s_j$ for all $s_j \in \mathcal{S}_i$ if $|\mathcal{S}_i| \geq 1$ and abort otherwise

Figure 10: Construction of TNIKE = $\{\text{Gen}_\lambda, \text{Share}_\lambda\}$.

Let $\mathcal{C}_1$ (respectively, $\mathcal{C}_2$) be the set of all word-RAM families such that for each $\mathcal{F} = \{f_\lambda\} \in \mathcal{C}_1$ and each $\lambda \in \mathbb{N}$, f_λ runs in time $\tilde{O}(\ell\lambda^2 + q\lambda^k)$ (respectively, $\tilde{O}((\ell\lambda^k)^{1-\delta})$) for some constant $\delta > 0$, where $\ell = \lambda^{\Omega(1)}$, $\ell = \text{poly}(\lambda)$, and $q < \ell$. Let $\bar{n}_p = 2^{\lceil \log_2 n_p \rceil}$, $\epsilon_1 = (1 - (q/\ell)^{n_p-1})^q + 2\ell n_p \lambda^k / R$ and

$$\epsilon_2 \leq 201/200 + \ell(2\bar{n}_p - 1) \binom{k}{2} 4^{\binom{k}{2}} 3\lambda^k / R - (1 - (q-1)/\ell)^{n_p} + 2n_p(q-1+\ell)\lambda^k / R.$$

We have the following theorem.

Theorem 4.8 *If the strong Zero- k -Clique- R hypothesis holds, then for any constant n_p , TNIKE is a $\mathcal{C}_1$ - n_p -NIKE with ϵ_1 -correctness and $\mathcal{C}_2$ - ϵ_2 -weak security.*

Proof. Complexity. Recall that Generate_λ runs in time $O(\lambda^2)$, and the check by brute forcing takes time $O(\lambda^k)$. Since all parties run Generate_λ for ℓ times and run check for q times, its total running time is $\tilde{O}(\ell\lambda^2 + q\lambda^k)$.

Correctness. We first prove that the probability that the protocol aborts is at most $(1 - (q/\ell)^{n_p-1})^q$. Without loss of generality, we assume that the indices in $\mathcal{S}_{n_p}$ are independently sampled from $[\ell]$ with replacement. Define the indicator Y_x such that $Y_x = \begin{cases} 1 & x \in \cap_{i=1}^{n_p-1} \mathcal{S}_i \\ 0 & \text{otherwise} \end{cases}$. For any $x \xleftarrow{\$} [\ell]$ we have

$$\Pr[Y_x = 1] = \Pr[x \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{n_p-1}] = \Pr[x \in \mathcal{S}_1] \cdots \Pr[x \in \mathcal{S}_{n_p-1}] = (q/\ell)^{n_p-1}$$

i.e., $\Pr[Y_x = 0] = 1 - (q/\ell)^{n_p-1}$. Moreover, for $x_1, \dots, x_q \xleftarrow{\$} [\ell]$ for any q , we have

$$\begin{aligned} \Pr[Y_{x_1} = 1 \wedge \dots \wedge Y_{x_q} = 1] &= \Pr[x_1, \dots, x_q \in \cap_{i=1}^{n_p-1} \mathcal{S}_i] \\ &= \prod_{i=1}^{n_p-1} \Pr[x_1, \dots, x_q \in \mathcal{S}_i] \quad (\because \text{independence of } \mathcal{S}_1, \dots, \mathcal{S}_{n_p-1}) \\ &= \prod_{i=1}^{n_p-1} (\Pr[x_1 \in \mathcal{S}_i] \dots \Pr[x_q \in \mathcal{S}_i]) \quad (\because \text{independence of } \{x_j\}_{j \in [q]}) \\ &= \Pr[x_1 \in \cap_{i=1}^{n_p-1} \mathcal{S}_i] \dots \Pr[x_q \in \cap_{i=1}^{n_p-1} \mathcal{S}_i] = \Pr[Y_{x_1} = 1] \dots \Pr[Y_{x_q} = 1]. \end{aligned}$$

Hence, $\{Y_{x_j}\}_{j \in [q]}$ are independent.

Since the intersection of $\mathcal{S}_1, \dots, \mathcal{S}_{n_p}$ only becomes larger when variables in $\mathcal{S}_{n_p}$ are sampled without replacement, the probability that $\cap_{i=1}^{n_p} \mathcal{S}_i = \emptyset$ (i.e., which is the probability that some user aborts) is no larger than

$$\Pr[\wedge_{i=1}^q Y_{x_i} = 0] = \prod_{j=1}^q \Pr[Y_{x_j} = 0] = (1 - (q/\ell)^{n_p-1})^q.$$

Moreover, the event that the n_p parties agree on the same key without aborting occurs if and only if $|\cap_{i=1}^{n_p} \mathcal{S}_i| > 0$ and no instance generated by $\text{Generate}_\lambda(R, k, 0)$ has a solution, which happens with probability

$$\Pr[\wedge_{i=1}^q Y_{x_i} = 0] + \Delta((I_i^j)_{j \in [\ell], i \in [n_p]}, (\mathcal{D}_{b_i^j}[R])_{j \in [\ell], i \in [n_p]}),$$

where $I_i^j \xleftarrow{\$} \text{Generate}_\lambda(R, k, b_i^j)$, $b_i^j = 1$ if $j \in \mathcal{S}_i$, and $b_i^j = 0$ otherwise. Since $\Pr[\wedge_{i=1}^q Y_{x_i} = 0] = (1 - (q/\ell)^{n_p-1})^q$, and

$$\Delta((I_i^j)_{j \in [\ell], i \in [n_p]}, (\mathcal{D}_{b_i^j}[R])_{j \in [\ell], i \in [n_p]}) = 2\ell n_p \lambda^k / R$$

according to Theorem 4.5, the correctness error probability is $\epsilon_1 = (1 - (q/\ell)^{n_p-1})^q + 2\ell n_p \lambda^k / R$.

$\mathcal{C}_2$ - ϵ -weak security. Let λ be the security parameter, and $\mathcal{A} = \{\text{adv}_\lambda\} \in \mathcal{C}_2$ be any adversary against the $\mathcal{C}_2$ - ϵ -weak security of **TNIKE**. The proof proceeds via a sequence of hybrid games as in Figure 11.

G₀. This is the original security game where adv_λ receives $(\text{pk}_i)_{i \in [n_p]}$ where $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\perp)$ for all $i \in [n_p]$, and outputs a session key key.

G₁. **G₁** and **G₀** only differ in the way that $(I_i^j)_{i \in [n_p], j \in [\ell]}$ are generated, namely, every instance I_i^j in pk_i is generated as $I_i^j \xleftarrow{\$} \mathcal{D}_0[R]$ (or $I_i^j \xleftarrow{\$} \mathcal{D}_1[R]$) rather than $I_i^j \xleftarrow{\$} \text{Generate}_\lambda(R, k, 0)$ (or $I_i^j \xleftarrow{\$} \text{Generate}_\lambda(R, k, 1)$) for all $i \in [n_p]$ and $j \in [\ell]$.

Lemma 4.9 $|\Pr[\mathcal{G}_1^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathcal{G}_0^{\text{adv}_\lambda} \Rightarrow 1]| \leq 2n_p \ell \lambda^k / R$.

This lemma follows immediately from Theorem 4.5.

G₂. **G₂** and **G₁** differ in the way that $(\text{sk}_i)_{i \in [n_p]}$ are generated, namely, for each $i \in [n_p]$, $\text{sk}_i = \{i^*\} \cup \mathcal{S}_i$ where $i^* \xleftarrow{\$} [\ell]$, $\mathcal{S}_i \subset [\ell]$, and $|\mathcal{S}_i| = q - 1$, and aborts if $|\text{sk}_i| < q$ rather than $\text{sk}_i = \mathcal{S}_i$ where $\mathcal{S}_i \subset [\ell]$ and $|\mathcal{S}_i| = q$.

Lemma 4.10 $|\Pr[\mathcal{G}_2^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathcal{G}_1^{\text{adv}_\lambda} \Rightarrow 1]| \leq 1 - (1 - (q - 1)/\ell)^{n_p}$.

Proof. We define **Bad** as the event that **G₂** aborts. Note that **Bad** does not occur as long as we avoid the index with one solution when planting $q - 1$ indices in each list. Let s_i^j be the j th variable in $\mathcal{S}_i$, for each $i \in [n_p]$ and $j \in [q - 1]$, we have $\Pr[\text{Bad}] = 1 - \prod_{i=1}^{n_p} \Pr[s_i^1 \neq i^* \wedge \dots \wedge s_i^{q-1} \neq i^*] \leq 1 - (1 - (q - 1)/\ell)^{n_p}$.

Since the view of adv_λ is identical to its view in **G₁** when **Bad** does not occur in **G₂**, we have

$$|\Pr[\mathcal{G}_2^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[\mathcal{G}_1^{\text{adv}_\lambda} \Rightarrow 1]| \leq \Pr[\text{Bad}] \leq 1 - (1 - (q - 1)/\ell)^{n_p},$$

completing this part of the proof. $\square$

G₃. **G₃** and **G₂** only differ in the way that $\{I_i^s\}_{i \in [n_p], s \in \mathcal{S}_i}$ are generated, namely, for each $i \in [n_p]$ and each $s \in \mathcal{S}_i$, $I_i^s \xleftarrow{\$} \text{Generate}_\lambda(R, k, 1)$ rather than $I_i^s \xleftarrow{\$} \mathcal{D}_1[R]$.

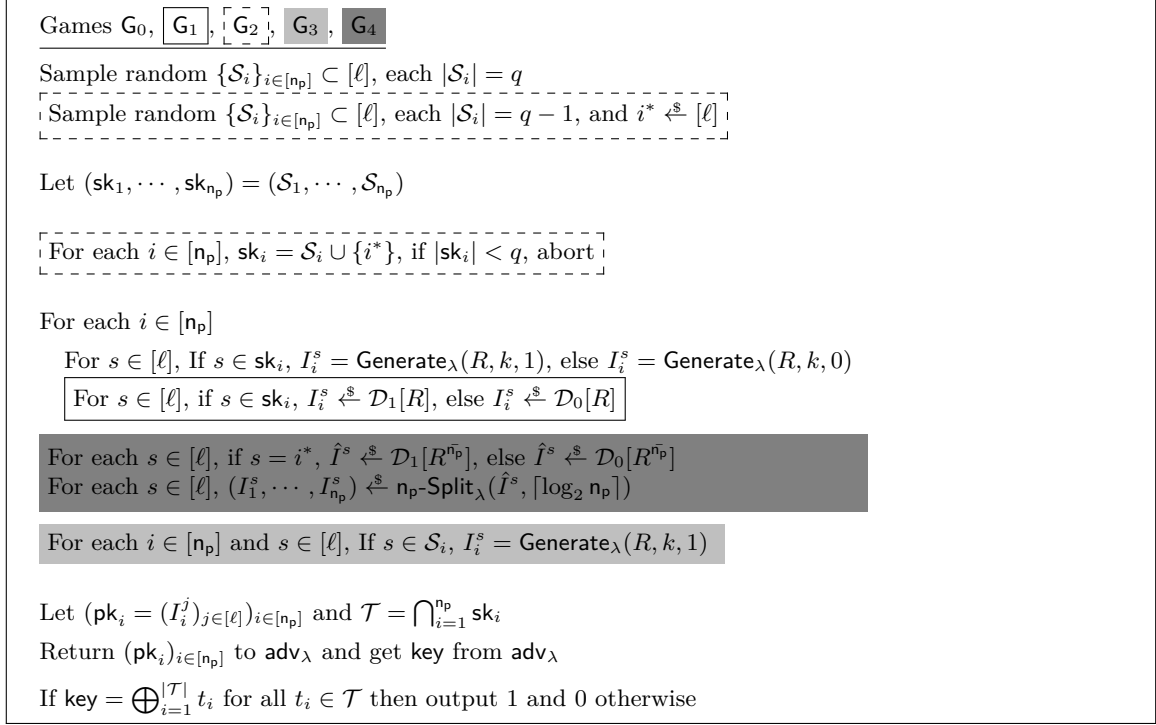


Figure 11: The challengers in G_0 , G_1 , G_2 , G_3 , and G_4 .

Lemma 4.11 $|\Pr[G_3^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[G_2^{\text{adv}_\lambda} \Rightarrow 1]| \leq 2n_p(q-1)\lambda^k/R$.

This lemma follows immediately from Theorem 4.5.

G_4 . In G_4 , $(I_i^j)_{i \in [n_p]}$ are generated by $\text{n}_p\text{-Split}_\lambda(\hat{I}^j, \lceil \log_2 n_p \rceil)$ for all j instead.

Here, note that $\text{n}_p\text{-Split}_\lambda$ returns $m = \binom{k}{2}^{\lceil \log_2 n_p \rceil}$ instance lists every time, and we can check the correctness of each instance in the list by brute forcing. When ℓ is polynomial in λ , the time required to check by brute forcing a single instance is polynomially smaller than the time needed to solve the entire list instance. We will need to check m of these instances by brute forcing, one for each of the m produced n_p lists. Since $m = \binom{k}{2}^{\lceil \log_2 n_p \rceil}$ is constant and $\ell = \lambda^{\Omega(1)}$, we have $m/\ell = \lambda^{-\Omega(1)}$. Therefore, the total time required for all the checks by brute forcing is polynomially smaller than the time needed to solve a single list instance, and we only focus on the instance list that is verified by this check.

Lemma 4.12 $|\Pr[G_4^{\text{adv}_\lambda} \Rightarrow 1] - \Pr[G_3^{\text{adv}_\lambda} \Rightarrow 1]| \leq \ell(2n_p - 1)\binom{k}{2}4^{\binom{k}{2}}3\lambda^k/R$.

By Theorem 4.7 and a union bound, Theorem 4.12 immediately follows.

Lemma 4.13 $\Pr[G_4^{\text{adv}_\lambda} \Rightarrow 1] \leq 1/200$.

Proof. We construct an adversary $\mathcal{B} = \{\text{advb}_\lambda\}$ in the game described in Theorem 4.3 as follows.

advb_λ runs in exactly the same way as the challengers in G_4 except that $(\hat{I}^1, \dots, \hat{I}^\ell)$ is generated by its own challenger as $\hat{I}^{i^*} \leftarrow \mathcal{D}_1[R^{n_p}]$ for $i^* \leftarrow [\ell]$ and $\hat{I}^j \leftarrow \mathcal{D}_0[R^{n_p}]$ for all $j \in [\ell] \setminus \{i^*\}$. When adv_λ returns key, advb_λ computes $\mathcal{T} = \bigcap_{i=1}^{n_p} \mathcal{S}_i$ and returns $i^* = \text{key} \oplus \bigoplus_{i=1}^{|\mathcal{T}|} t_i$.

Recall that $\text{n}_p\text{-Split}_\lambda$ runs in time $O(\lambda^{k(1-\delta_{sp})})$ for some constant $\delta_{sp} > 0$. Since advb_λ runs adv_λ once, $\text{n}_p\text{-Split}_\lambda$ for ℓ times, Generate_λ for $n_p \cdot q$ times, and check for $\binom{k}{2}^{\lceil \log_2 n_p \rceil}$ times, its total running time is $\tilde{O}((\ell\lambda^k)^{1-\delta} + \ell\lambda^{k(1-\delta_{sp})} + n_p q \lambda^2 + \binom{k}{2}^{\lceil \log_2 n_p \rceil} \lambda^k) = \tilde{O}((\ell\lambda^k)^{1-\delta'})$ for some $\delta' > 0$.

Since the view of adv_λ is identical to its view in G_4 , the advantage of advb_λ is exactly $\Pr[G_4^{\text{adv}_\lambda} \Rightarrow 1]$, which is no more than $1/200$, according to Theorem 4.3. $\square$

Putting all the above together, we have

$$\Pr[G_0^{\text{adv}\lambda} \Rightarrow 1] \leq 201/200 + \epsilon_{\text{split}} - (1 - (q-1)/\ell)^{n_p} + 2n_p(q-1+\ell)\lambda^k/R,$$

where $\epsilon_{\text{split}} = \ell(2\bar{n}_p - 1)\binom{k}{2}4^{\binom{k}{2}}3\lambda^k/R$, completing the proof of Theorem 4.8. $\square$

Example with asymptotic complexity. Let $\ell = \lambda^{n_p}$, $q = \lambda^{n_p-1} \log \lambda$, and $R = \lambda^{n_p k}$. We have

$$\epsilon_1 \leq (1 - (q/\ell)^{n_p-1})^q + 2\ell n_p \lambda^k/R = (1 - ((\lambda^{n_p-1} \log \lambda)/\lambda^{n_p})^{n_p-1})^{\lambda^{n_p-1} \log \lambda} + 2n_p \lambda^{n_p+k}/\lambda^{n_p k},$$

which approaches

$$e^{-\log^{n_p} \lambda} + O(1/\lambda^{kn_p-k-n_p}) = O(1/\lambda^{kn_p-k-n_p})$$

when λ is sufficiently large, and

$$\epsilon_2 \leq 1/200 + \tilde{O}(\lambda^{-1}) + \tilde{O}(\lambda^{n_p+k-kn_p}).$$

In this case, the scheme is computable within time $\tilde{O}(\lambda^{n_p+k-1})$ and secure against adversaries running in time $\tilde{O}(\lambda^{n_p+k-\Omega(1)})$. When n_p is fixed, setting $k = 3$ provides better bounds in our instantiation.

Concrete example. We now give a concrete example of the parameters. Let $\lambda = 2^{16}$, $\ell = 2^{48}$, $q = 2^{36}$, $R = 2^{144}$, $n_p = 3$, and $k = 3$, since $(1 - (q/\ell)^{n_p-1})^q < 10^{-16}$, $2\ell n_p \lambda^k/R = 6/2^{48} < 2.14 \times 10^{-14}$ and $201/200 - (1 - (q-1)/\ell)^{n_p} < 1/200 + 1/1000 = 3/500$, we have $\epsilon_1 < 2.15 \times 10^{-14}$ and $\epsilon_2 \leq 3/500 + 1/500 < 1/125$.

Full security. While TNIKE has an $O(\log \lambda)$ -bit weakly secure key, the Goldreich–Levin transformation in Figure 12 outputs a one-bit key key^* . This gives a fully secure scheme with the parameter loss stated in Theorem 4.15. Before giving our fully secure construction, we recall the following Chebyshev bound.

Theorem 4.14 (Chebyshev bound). *For any $\epsilon > 0$, $b \in \{0, 1\}$, and $q \in \mathbb{N}$, let $\{x_i\}_{i \in [q]}$ be pairwise-independent, 0/1-random variables such that*

$$\Pr[x_i = b] \geq 1/2 + \epsilon$$

for all $i \in [q]$. Let $x = \begin{cases} 1 & \text{if } \sum_{i=1}^q x_i \geq q/2 \\ 0 & \text{if } \sum_{i=1}^q x_i < q/2 \end{cases}$. Then

$$\Pr[x \neq b] \leq 1/(4\epsilon^2 q).$$

Let $\mathcal{C}_1$ (respectively, $\mathcal{C}_2$) be the set of all word-RAM families such that for each $\mathcal{F} = \{f_\lambda\} \in \mathcal{C}_1$ and each $\lambda \in \mathbb{N}$, f_λ runs in time $\tilde{O}(T_1(\lambda))$ (respectively, $\tilde{O}(T_2(\lambda))$) for any constant $n_p \geq 2, k \geq 3$. Let $\epsilon = 1/\lambda^{o(1)}$ and TNIKE = $(\text{Gen}_\lambda, \text{Share}_\lambda)$ be a $\mathcal{C}_1$ - n_p -NIKE that satisfies $\mathcal{C}_2$ - ϵ -weak security with the shared key size $Q(\lambda) = \Theta(\log(\lambda))$, we construct $\text{TNIKE}^* = (\text{Gen}_\lambda^*, \text{Share}_\lambda^*)$ satisfying $\mathcal{C}_2$ - $1/\lambda^{o(1)}$ -security as in Figure 12.

Theorem 4.15 *If TNIKE is a $\mathcal{C}_1$ - n_p -NIKE with $\mathcal{C}_2$ - ϵ -weak security, then TNIKE^* is a $\mathcal{C}_1$ - n_p -NIKE with $\mathcal{C}_2$ - $1/\lambda^{o(1)}$ -security.*

$\text{Gen}_\lambda^*(\perp)$: $(\text{pk}, \text{sk}) \xleftarrow{\\$} \text{Gen}_\lambda(\perp)$ $\mathbf{v} \xleftarrow{\\$} \{0, 1\}^{Q(\lambda)}$ Return $(\text{pk}^* = (\text{pk}, \mathbf{v}), \text{sk}^* = \text{sk})$	$\text{Share}_\lambda^*(i, \text{sk}_i^*, (\text{pk}_j^*)_{j \in [n_p]})$: Parse $\text{pk}_j^* = (\text{pk}_j, \mathbf{v}_j)$ for all $j \in [n_p]$ $\text{key} = \text{Share}_\lambda(i, \text{sk}_i^*, (\text{pk}_j)_{j \in [n_p]})$ If $\text{key} = \perp$, return $\perp$ Return $\text{key}^* = \text{key} \cdot \mathbf{v}_{n_p}$
---	--

Figure 12: Definition of $\text{TNIKE}^* = \{\text{Gen}_\lambda^*, \text{Share}_\lambda^*\}$ with $\mathcal{C}_2$ - $1/\lambda^{o(1)}$ -security.

Proof. Let $m = 2Q(\lambda)/\epsilon^2$, $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\perp)$ for all $i \in [n_p]$, $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, \text{pk}_{[n_p]})$, $\mathbf{v}_{n_p} \xleftarrow{\$} \{0, 1\}^{Q(\lambda)}$, and $\mathcal{A} = \{\text{adv}_\lambda\} \in \mathcal{C}_2$ be any adversary breaking the $\mathcal{C}_2$ - ϵ -security of TNIKE^* . We construct an adversary $\mathcal{B} = \{b_\lambda\} \in \mathcal{C}_2$ breaking the $\mathcal{C}_2$ - ϵ -weak security of TNIKE .

Firstly, b_λ chooses $\log(m)$ pairs $(\sigma_1, r_1), \dots, (\sigma_{\log(m)}, r_{\log(m)}) \xleftarrow{\$} \{0, 1\} \times \{0, 1\}^{Q(\lambda)}$. Then, for every nonempty subset $\mathcal{X} \subseteq [\log(m)]$, b_λ computes $r_{\mathcal{X}} = \bigoplus_{i \in \mathcal{X}} r_i$ and $\sigma_{\mathcal{X}} = \bigoplus_{i \in \mathcal{X}} \sigma_i$. For every $i \in [Q(\lambda)]$, b_λ runs

$$\text{key}_i^{\mathcal{X}} = \sigma_{\mathcal{X}} \oplus \text{adv}_\lambda(\{\text{pk}_i\}_{i \in [n_p]}, r_{\mathcal{X}} \oplus e^i),$$

where e^i denotes the $Q(\lambda)$ -bit string with 1 in the i 'th position and 0 everywhere else, and sets key_i^* as the majority of $\text{key}_i^{\mathcal{X}}$. Finally, b_λ outputs $\text{key}^* = \text{key}_1^* || \dots || \text{key}_{Q(\lambda)}^*$. Let

$$\mathcal{S} = \{\text{key} \mid \Pr[\text{adv}_\lambda(\{\text{pk}_i\}_{i \in [n_p]}, \mathbf{v}_{n_p}) = \text{key}] > 1/2 + \epsilon/2\}.$$

A quick calculation yields $|\mathcal{S}| > 2^{Q(\lambda)-1}\epsilon$. Since $\{\sigma_i\}_{i \in [\log(m)]}$ and $\{r_i\}_{i \in [\log(m)]}$ are sampled uniformly at random, we have

$$\Pr[(\sigma_i = r_i \cdot \text{key})_{i \in [\log(m)]}] = 1/m.$$

Notice that the set $\{r_{\mathcal{X}}\}_{\mathcal{X} \subseteq [\log(m)]}$ are pairwise independent, the probability that the majority of bits is incorrect is

$$\Pr[(\text{key}_i^* \neq \text{key}_i) \mid (\sigma_j = \text{key} \cdot r_j)_{j \in [\log(m)]} \wedge (\text{key} \in \mathcal{S})] \leq 1/(m\epsilon^2),$$

for each $i \in [Q(\lambda)]$ by Chebyshev bound. Putting all these together we have

$$\begin{aligned} \Pr[b_\lambda(\{\text{pk}_i\}_{i \in [n_p]}) = \text{key}] &\geq \Pr[\text{key} \in \mathcal{S}] \cdot \Pr[b_\lambda(\{\text{pk}_i\}_{i \in [n_p]}) = \text{key} \mid \text{key} \in \mathcal{S}] \\ &= \epsilon/(2m) \cdot \Pr[\forall i \in [Q(\lambda)], (\text{key}_i^* = \text{key}_i) \mid \text{key} \in \mathcal{S}, (\sigma_j = \text{key} \cdot r_j)_{j \in [\log(m)]}] \\ &= \epsilon/(2m)(1 - \Pr[\exists i \in [Q(\lambda)] \text{ s.t. } \text{key}_i^* \neq \text{key}_i \mid \text{key} \in \mathcal{S}, (\sigma_j = \text{key} \cdot r_j)_{j \in [\log(m)]}]) \\ &\geq \epsilon/(2m)(1 - Q(\lambda) \Pr[\text{key}_i^* \neq \text{key}_i \mid (\sigma_j = \text{key} \cdot r_j)_{j \in [\log(m)]}, \text{key} \in \mathcal{S}]) \\ &\geq \epsilon/(2m)(1 - Q(\lambda)/(m\epsilon^2)) = \epsilon/(2m) - Q(\lambda)/(2m^2\epsilon). \end{aligned}$$

Since $Q(\lambda) = \Theta(\log(\lambda))$ and $\epsilon = 1/\lambda^{o(1)}$, the probability that b_λ succeeds is at least $\epsilon^3/(8\Theta(\log(\lambda))) = 1/\lambda^{o(1)}$. Since adv_λ runs in time $\tilde{O}(T_2(\lambda))$ and $m = \Theta(\log \lambda)\lambda^{o(1)}$, $\{b_\lambda\}$ runs in time $\tilde{O}(\Theta(\log \lambda)) \cdot m \cdot T_2(\lambda) = \tilde{O}(T_2(\lambda))$, completing the proof of Theorem 4.15. $\square$

Extension to IB-NIKE. Similar to our construction in the bounded-parallel-time model given in Section 3.2, we can show that our construction in this section satisfies extendability (see Theorem 2.2), and thus can be converted into a BPRF and an IB-NIKE in the same fine-grained setting. We refer the reader to Section A for the full details.

5 Multi-Party Key Exchange in the Bounded Storage Model

In this section, we extend the two-party NIKE in the BSM [?] to a multi-party one and show the optimality of our construction up to constant factors within the natural class of schemes satisfying extendability and local key independence by giving a lower bound for the users' storage consumption. All algorithms in this section are considered as streaming word-RAMs except for the storage function applied by the adversary, which can be any unbounded function with limited output size.

5.1 Our Construction

Pairwise independence. We first recall the definitions of uniformly and approximately pairwise independence, (approximately) strongly 2-universal hash function family, and 2-universal hash function family, and recall their instantiations in Figure 13.¹

Definition 5.1 (Uniform pairwise independence). A sequence of random variables $x_1, \dots, x_q$ with alphabet $\mathcal{Y}$ is uniformly pairwise independent if for any $1 \leq i < j \leq q$ and any $\hat{y}_1, \hat{y}_2 \in \mathcal{Y}$, we have $\Pr[x_i = \hat{y}_1 \wedge x_j = \hat{y}_2] = 1/|\mathcal{Y}|^2$.

¹The approximate pairwise independence and approximate 2-universal hash function family are defined in [?, ?] in an implicit way.

Definition 5.2 (Approximate pairwise independence [?, ?]). A sequence of random variables $x_1, \dots, x_q$ with alphabet $\mathcal{Y}$ is approximately pairwise independent if for any $1 \leq i < j \leq q$ and any distinct $\hat{y}_1, \hat{y}_2 \in \mathcal{Y}$, we have $\Pr[x_i = \hat{y}_1 \wedge x_j = \hat{y}_2] = 1/(|\mathcal{Y}|(|\mathcal{Y}| - 1))$.

Definition 5.3 (Strongly 2-universal hash function family). A set $\mathcal{H}$ of functions with domain $\mathcal{X}$ and range $\mathcal{Y}$ is said to be a *strongly 2-universal hash function family* if for any distinct $x_1, x_2 \in \mathcal{X}$, $h(x_1)$ and $h(x_2)$ are uniformly pairwise independent for $h \leftarrow^{\$} \mathcal{H}$.

Definition 5.4 (Approximate strongly 2-universal hash function [?, ?]). The definition of *approximate strongly 2-universal hash function family* is exactly the same as Theorem 5.3 except that we replace “uniformly pairwise independent” by “approximate pairwise independent”.

Definition 5.5 (2-universal hash function family). A set $\mathcal{H}$ of functions with domain $\mathcal{X}$ and range $\mathcal{Y}$ is said to be a *2-universal hash function family* if for any distinct $x_1, x_2 \in \mathcal{X}$, we have $\Pr[g(x_1) = g(x_2)] \leq 1/|\mathcal{Y}|$, where $g \leftarrow^{\$} \mathcal{H}$.

$$\begin{aligned} \mathcal{F}_n &= \{f(x) = a_1 \cdot x + a_0 \mid a_0, a_1 \in GF(n)\} \\ \mathcal{H}_n &= \{h(x) = a_1 \cdot x + a_0 \mid a_0, a_1 \in GF(n) \wedge a_1 \neq 0\} \\ \mathcal{G}_{\lambda, r} &= \{g(x) = \text{MSB}_r(a \cdot x) \mid a \in GF(2^\lambda)\} \end{aligned}$$

Figure 13: The definitions of $\mathcal{F}_n$, $\mathcal{H}_n$, and $\mathcal{G}_{\lambda, r}$. In this figure, all operations involved in f and h (respectively, g) are within the field $GF(n)$ (respectively, $GF(2^\lambda)$). One can see that the outputs of any function in $\mathcal{H}_n$ with distinct inputs must be all distinct.

Lemma 5.6 ([?, ?]). For any finite field $GF(n)$ and any $\lambda, r \in \mathbb{N}$, $\mathcal{F}_n$ (respectively, $\mathcal{H}_n$) is a strongly 2-universal hash function family (respectively, approximate strongly 2-universal hash function family), and $\mathcal{G}_{\lambda, r}$ is a 2-universal hash function family.

Parameters. Next we follow [?] to define several parameters. Let $\lambda \in \mathbb{N}$ be the security parameter and n_p be the number of parties. Let m_a, n be integers such that $m_a < n$. We define q , ϵ_1 , ϵ_2 , r , and θ satisfying the following equations:

- $\delta = \min\{0.9453, (1/n)(n - m_a - \log(1/\epsilon_1))\}$,
- ρ such that $H_b(\rho) + \rho \log(1/\delta) + 1/n = \delta$,
- $\lambda = (q/2) \cdot (q/n)^{n_p-1} = \lfloor (\rho \epsilon_2^2 - \rho 2^{-\rho n \log(1/\delta)-2})^{-1} \rfloor$,
- $r = \lfloor \log \theta + \rho \lambda / 2 - 1 \rfloor$,

where H_b is the *binary entropy function* defined as $H_b(\rho) = -\rho \log \rho - (1 - \rho) \log(1 - \rho)$. As noted in [?], when $n \gg m_a$, we have $\delta = 0.9453$, $\rho = 1/3$, $\lambda \approx 3/\epsilon_2^2$, and $r \approx \lambda/6$.

Before giving our construction, we recall the following two theorems. The first one is the main theorem in [?] for privacy amplification, and the second one is a direct combination of Chebyshev’s inequality and Markov’s inequality.

Theorem 5.7 ([?]). Let $\lambda \in \mathbb{N}$ and $GF(n)$ be a finite field. Let $\mathcal{A}$ be any adversary with the memory bound $m_a < n$, $\text{urs} \leftarrow^{\$} \{0, 1\}^n$, $\mathbf{z} = \mathcal{A}(\text{urs})$, $g \leftarrow^{\$} \mathcal{G}_{\lambda, r}$, $\mathbf{s} = (s_i)_{i \in [\lambda]}$ be a sequence of approximate pairwise independent indices over $GF(n)$, and $\text{key} = g(\text{urs}_{\mathbf{s}})$. There exists a security event $\mathbf{E}$ such that

$$\exists \mathbf{E} : \Pr[\mathbf{E}] \geq 1 - \epsilon_1 - \epsilon_2 \text{ and } I(\text{key}; g, \mathbf{z}, \mathbf{s} \mid \mathbf{E}) \leq \theta.^2$$

Theorem 5.8 For any $n \in \mathbb{N}$, let $(t_i)_{i \in [n]}$ be a sequence of pairwise independent variables. Then for any $a > 0$, we have

$$\Pr\left[\left|\sum_{i=1}^n t_i - \sum_{i=1}^n \mathbb{E}[t_i]\right| \geq a\right] \leq \frac{\sum_{i=1}^n \text{Var}[t_i]}{a^2}.$$

Construction. Let $\mathcal{H}_n$ be the approximately strongly 2-universal hash function family, and $\mathcal{G}_{\lambda, r}$ be the 2-universal hash function family, as defined in Figure 13. Let $\mathcal{C}_1$ be the class of streaming word-RAM families with memory bounds $4 \log n + \lambda + q$, where $q = n \sqrt[3]{2\lambda/n}$. Throughout this construction, the parameters satisfy $q \leq n$. Our construction of multi-party NIKE in the BSM is defined as in Figure 14.

²The original theorem asserts that $I(\text{key}; g \mid \mathbf{z}, \mathbf{s}, \mathbf{E}) \leq \theta$, while their proof clearly implies our version.

<u>$\text{Gen}_\lambda(\text{urs} \in \{0, 1\}^n)$:</u> Set $\text{pk}_i = (h_i, g_i)$ where $h_i \xleftarrow{\\$} \mathcal{H}_n$ and $g_i \xleftarrow{\\$} \mathcal{G}_{\lambda, r}$ Set $\text{sk}_i = (h_i, \text{urs}_{\mathcal{S}_i})$ where $\mathcal{S}_i$ is the set of indices in $(h_i(j))_{j \in [q]}$ Return $(\text{pk}_i, \text{sk}_i)$ <u>$\text{Share}_\lambda(i, \text{sk}_i, (\text{pk}_j)_{j \in [\mathfrak{n}_p]})$</u> Parse $\text{sk}_i = (h_i, \text{urs}_{\mathcal{S}_i})$ Remove bits in $\text{urs}_{\mathcal{S}_i}$ with indices not in $\mathcal{S}_1 \cap \dots \cap \mathcal{S}_{\mathfrak{n}_p}$ by computing $(h_i(j))_{i \in [\mathfrak{n}_p], j \in [q]}$ in a streaming manner For each $j \in [q]$, let $u_j = h_{\mathfrak{n}_p}(j)$ if $h_{\mathfrak{n}_p}(j) \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{\mathfrak{n}_p}$, and let $u_j = \perp$ otherwise Let $\mathbf{s}$ be obtained from $(u_j)_{j \in [q]}$ by removing all $\perp$'s while preserving the order of j Abort the whole protocol if $\mathbf{s} < \lambda$ Return $\text{key} = g_{\mathfrak{n}_p}(\text{urs}_{\mathbf{s}_\lambda})$ where $\mathbf{s}_\lambda$ denotes the first λ entries of $\mathbf{s}$
--

Figure 14: The definition of BSMKE. Recall that for any set $\mathcal{S}$, $\text{urs}_{\mathcal{S}}$ denotes the sequence of all bits in urs with indices (ordered lexicographically) in $\mathcal{S}$. For any sequence $\mathbf{s}$, $\text{urs}_{\mathbf{s}}$ follows the order of $\mathbf{s}$.

Theorem 5.9 *BSMKE is a $\mathcal{C}_1$ - $\mathfrak{n}_p$ -NIKE with $2/\lambda$ -correctness and $(\mathfrak{m}_a, \epsilon_1 + \epsilon_2 + 2/\lambda, \theta)$ -security in the BSM.*

Proof. Complexity. First we note that each $\text{P}_{i \in [\mathfrak{n}_p]}$ consumes $4 \log n + \lambda + q$ bits of memory to store q bits in urs , a 2-universal hash function in $\mathcal{G}_{\lambda, r}$, and at most two strongly 2-universal hash functions in $\mathcal{H}_n$ for comparison at any point in the protocol.

Correctness. To prove correctness, it suffices to show that the protocol aborts with probability at most $2/\lambda$.

Let $\mathcal{S}_i = \{h_i(j) : j \in [q]\}$ for $i \in [\mathfrak{n}_p]$, and define

$$Y_j = \begin{cases} 1 & \text{if } h_{\mathfrak{n}_p}(j) \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{\mathfrak{n}_p-1}, \\ 0 & \text{otherwise} \end{cases} \quad \text{and } Y = \sum_{j=1}^q Y_j.$$

Since $h_{\mathfrak{n}_p} \xleftarrow{\$} \mathcal{H}_n$ is injective on the q queried points, Y is exactly the number of distinct common indices in $\mathcal{S}_1 \cap \dots \cap \mathcal{S}_{\mathfrak{n}_p}$. Thus the protocol aborts iff $Y < \lambda$.

For each fixed $a \in GF(n)$ and each $i \in [\mathfrak{n}_p - 1]$, $\Pr[a \in \mathcal{S}_i] = q/n$. Since $\mathcal{S}_1, \dots, \mathcal{S}_{\mathfrak{n}_p-1}$ are independent and $h_{\mathfrak{n}_p}(j)$ is uniform and independent of them,

$$\mathbb{E}[Y] = \sum_{j=1}^q \mathbb{E}[Y_j] = \sum_{j=1}^q \Pr[h_{\mathfrak{n}_p}(j) \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{\mathfrak{n}_p-1}] = \sum_{j=1}^q \left(\frac{q}{n}\right)^{\mathfrak{n}_p-1} = q \left(\frac{q}{n}\right)^{\mathfrak{n}_p-1} = 2\lambda.$$

We next bound the variance. For $j \neq j'$,

$$\begin{aligned}
\mathbb{E}[Y_j Y_{j'}] &= \Pr[Y_j = Y_{j'} = 1] \\
&= \sum_{\substack{a, b \in GF(n) \\ a \neq b}} \Pr[h_{n_p}(j) = a, h_{n_p}(j') = b] \Pr[a, b \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{n_p-1}] \\
&= \sum_{\substack{a, b \in GF(n) \\ a \neq b}} \frac{1}{n(n-1)} \Pr[a, b \in \mathcal{S}_1 \cap \dots \cap \mathcal{S}_{n_p-1}] \quad (\because \text{approximate pairwise independence of } h_{n_p}) \\
&= \sum_{\substack{a, b \in GF(n) \\ a \neq b}} \frac{1}{n(n-1)} \prod_{i=1}^{n_p-1} \Pr[a, b \in \mathcal{S}_i] \quad (\because \text{independence of } (\mathcal{S}_i)_{i=1}^{n_p-1}) \\
&= \sum_{\substack{a, b \in GF(n) \\ a \neq b}} \frac{1}{n(n-1)} \left(\frac{q(q-1)}{n(n-1)} \right)^{n_p-1} \quad (\because \text{injectivity and uniformity of } \{h_i\}_{i=1}^{n_p-1}) \\
&= \left(\frac{q(q-1)}{n(n-1)} \right)^{n_p-1} \leq \left(\frac{q}{n} \right)^{2(n_p-1)} = \mathbb{E}[Y_j] \mathbb{E}[Y_{j'}].
\end{aligned}$$

As a result, we have

$$\text{Var}[Y] = \sum_{j=1}^q \text{Var}[Y_j] + 2 \sum_{1 \leq j < j' \leq q} (\mathbb{E}[Y_j Y_{j'}] - \mathbb{E}[Y_j] \mathbb{E}[Y_{j'}]) \leq \sum_{j=1}^q \text{Var}[Y_j] \leq \sum_{j=1}^q \mathbb{E}[Y_j] = 2\lambda.$$

By Chebyshev's inequality,

$$\Pr[Y < \lambda] \leq \Pr[|Y - \mathbb{E}[Y]| \geq \lambda] \leq \frac{\text{Var}[Y]}{\lambda^2} \leq \frac{2}{\lambda}.$$

Therefore, the protocol aborts with probability at most $2/\lambda$.

Security. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^{m_a}$ be any storage function applied by the adversary. Let $(h_i, g_i)_{i \in [n_p]}$ be the honestly generated public keys. Let $\mathbf{u} = (u_i)_{i \in [q]}$ be the sequence such that $u_i = h_{n_p}(i)$ if there are indices $i_1, \dots, i_{n_p-1}$ satisfying $h_1(i_1) = \dots = h_{n_p-1}(i_{n_p-1}) = h_{n_p}(i)$ and $u_i = \perp$ otherwise, and let $\mathcal{S}$ be the set of all indices of u_i restricted to $u_i \neq \perp$, i.e., $\mathbf{s} = \mathbf{u}_{\mathcal{S}}$ be the sequence $\mathbf{u}$ restricted to $u_i \neq \perp$. We have the following lemma.

Lemma 5.10 $\mathbf{s}$ is approximately pairwise independent.

Proof. Let $\hat{x}_1, \hat{x}_2$ be any distinct elements in $GF(n)$. Due to approximately pairwise independence, we have $\Pr[h_{n_p}(i) = \hat{x}_1 \wedge h_{n_p}(j) = \hat{x}_2] = \frac{1}{n(n-1)}$ for all $i, j \in [q]$, and $\Pr[\exists i_l, j_l : h_l(i_l) = \hat{x}_1 \wedge h_l(j_l) = \hat{x}_2] = \frac{q(q-1)}{n(n-1)}$ for all $l \in [n_p - 1]$. Hence, for any $i, j \in \mathbb{Z}_q$, we have

$$\begin{aligned}
\Pr[u_i = \hat{x}_1 \wedge u_j = \hat{x}_2] &= \Pr[\exists (i_l, j_l)_{l \in [n_p-1]} : h_1(i_1) = \dots = h_{n_p-1}(i_{n_p-1}) = \hat{x}_1 = h_{n_p}(i) \wedge \\
&\quad h_1(j_1) = \dots = h_{n_p-1}(j_{n_p-1}) = \hat{x}_2 = h_{n_p}(j)] \\
&= \Pr[h_{n_p}(i) = \hat{x}_1 \wedge h_{n_p}(j) = \hat{x}_2] \cdot \prod_{l=1}^{n_p-1} \Pr[\exists i_l, j_l : h_l(i_l) = \hat{x}_1 \wedge h_l(j_l) = \hat{x}_2] \\
&= \frac{1}{n(n-1)} \cdot \left(\frac{q(q-1)}{n(n-1)} \right)^{n_p-1},
\end{aligned}$$

and

$$\begin{aligned}
\Pr[u_i \neq \perp \wedge u_j \neq \perp] &= \Pr[\exists (i_l, j_l)_{l \in [n_p-1]} : h_1(i_1) = \dots = h_{n_p-1}(i_{n_p-1}) = h_{n_p}(i) \wedge \\
&\quad h_1(j_1) = \dots = h_{n_p-1}(j_{n_p-1}) = h_{n_p}(j)] \\
&= \prod_{l=1}^{n_p} \Pr[\exists i_l, j_l : h_l(i_l) = h_{n_p}(i) \wedge h_l(j_l) = h_{n_p}(j)] = \left(\frac{q(q-1)}{n(n-1)} \right)^{n_p-1},
\end{aligned}$$

i.e.,

$$\Pr[u_i = x_1 \wedge u_j = x_2 | u_i \neq \perp \wedge u_j \neq \perp] = \frac{\Pr[u_i = \hat{x}_1 \wedge u_j = \hat{x}_2]}{\Pr[u_i \neq \perp \wedge u_j \neq \perp]} = \frac{1}{n(n-1)}.$$

This completes the proof of Theorem 5.10. $\square$

By the correctness analysis above, we have $\Pr[\text{Good}] \geq 1 - 2/\lambda$. On **Good**, let $\mathbf{s}_\lambda$ denote the first λ entries of $\mathbf{s}$. By Theorem 5.10 and Theorem 5.7, there exists an event $\mathbf{E}_0$ such that

$$\Pr[\mathbf{E}_0 \mid \text{Good}] \geq 1 - \epsilon_1 - \epsilon_2 \text{ and } I(\text{key}; g_{n_p}, \mathbf{z}, \mathbf{s}_\lambda \mid \mathbf{E}_0, \text{Good}) \leq \theta,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\mathbf{z} = f(\text{urs})$, and $\text{key} = g_{n_p}(\text{urs}_{\mathbf{s}_\lambda})$. Let $\mathbf{E} = \mathbf{E}_0 \cap \text{Good}$. Then we have

$$\Pr[\mathbf{E}] \geq 1 - \epsilon_1 - \epsilon_2 - 2/\lambda.$$

The event $\mathbf{E}_0$ is chosen using only $(\text{urs}, g_{n_p}, \mathbf{z}, \mathbf{s}_\lambda)$ and not the remaining public-key components. Since $\mathbf{s}_\lambda$ is determined by $(h_i)_{i \in [n_p]}$ and $\text{key} = g_{n_p}(\text{urs}_{\mathbf{s}_\lambda})$, those remaining components give no additional information about key once $(g_{n_p}, \mathbf{z}, \mathbf{s}_\lambda, \mathbf{E})$ is fixed. Hence

$$I(\text{key}; (\mathbf{z}, (\text{pk}_i)_{i \in [n_p]}) \mid \mathbf{E}) = I(\text{key}; g_{n_p}, \mathbf{z}, \mathbf{s}_\lambda \mid \mathbf{E}) \leq \theta.$$

Thus, BSMKE satisfies $(\mathbf{m}_a, \epsilon_1 + \epsilon_2 + 2/\lambda, \theta)$ -security in the BSM.

Putting all the above together, Theorem 5.9 follows. $\square$

Example with asymptotic complexity. Let $n = \lambda^{n_p+1}$, $\mathbf{m}_a = \lambda^{n_p+1}/2$, $\epsilon_1 = 1/\lambda^{n_p}$, $\epsilon_2 = 1/\sqrt{\lambda}$, and $\theta = \text{negl}(\lambda)$. We have $r = O(\lambda)$. In this case, each $P_{i \in [n_p]}$ consumes about $q = n^{n_p/2\lambda/n} = O(\lambda^{n_p})$ bits of memory and the communication cost, i.e., the total size of the public keys, is $O(\lambda)$.

Concrete example. We now give a concrete example of the system based on the data given in [?]. Assume that all users have access to a 40 Gbit/s broadcast channel used for 2×10^5 seconds (about two days), we have $n = 8.6 \times 10^{15}$. Let $\mathbf{m}_a = 4.5 \times 10^{15}$. With $\theta = 10^{-20}$ and error probabilities $\epsilon_1 = 10^{-20}$ and $\epsilon_2 = 10^{-3}$, the parameters are $\delta = 0.476$, $\rho = 0.077$, $\lambda = 1.3 \times 10^7$, and $r = 5.0 \times 10^5$. Except with probability about 10^{-3} , Eve knows less than 10^{-20} bits about the 61 KByte session key. To obtain this r -bit session key in a 3-party protocol, each party consumes about 1.2×10^{13} bits of memory, and the communication consists of about 3.9×10^7 bits in total.

Extensions. Similar to our constructions in previous sections, our construction in the BSM can also be converted into a BPRF and a multi-party IB-NIKE. We refer the reader to Section A for the details.

5.2 Limitations of Multi-Party NIKes in the BSM

In this section, we show that for an NIKE treating any constant number of parties satisfying (restricted) extendability and local key independence, the entropy of the shared key conditioned on the public keys is bounded by roughly $\mathbf{m}_u^{n_p}/\mathbf{m}_a^{n_p-1}$, where $\mathbf{m}_u$ and $\mathbf{m}_a$ are the memory bounds for users and adversaries respectively. This suggests that $\mathbf{m}_u$ must be at least about $\sqrt[n_p]{\lambda \cdot \mathbf{m}_a^{n_p-1}}$ for sharing λ bits, and thus our multi-party NIKE in the BSM, which satisfies local key independence as we show later, achieves optimality up to constant factors within this natural class.

Before giving our negative result, we recall several lemmata and theorems below.

Lemma 5.11 For two random variables x, y , we have $H(x) \geq H(x \mid y)$.

Theorem 5.12 (Data processing inequality). For two random variables x, y and deterministic functions f, g , we have

$$H(f(x)) \leq H(x) \text{ and } I(f(x); g(y)) \leq I(x; y).$$

The same inequalities hold conditioned on any additional random variable.

Lemma 5.13 (Lemma 1 in [?]). Let $y, z, (z_i)_{i \in [n]}$ be a sequence of random variables such that conditioned on y , the variables $z, z_1, \dots, z_n$ are independent and identically distributed, i.e.,

$$\Pr[z = \hat{z} \wedge z_1 = \hat{z}_1 \wedge \dots \wedge z_n = \hat{z}_n \mid y = \hat{y}] = \Pr[z = \hat{z} \mid y = \hat{y}] \prod_{i=1}^n \Pr[z = \hat{z}_i \mid y = \hat{y}]$$

for all $y, z, (z_i)_{i \in [n]}$. Then $I(y; z \mid z_1, \dots, z_n) \leq H(y)/(n+1)$.

Theorem 5.14 (Theorem 1 in [9]). Let x, y, z, c_0, c, s, s' be random variables such that

$$H(s \mid x, c_0, c) = 0, \quad H(s' \mid y, c_0, c) = 0, \quad \Pr[s \neq s'] = 0,$$

and

$$I(x; y \mid z, c_0, c) \leq I(x; y \mid z, c_0).$$

Then

$$H(s) \leq I(x; y \mid z, c_0) + I(s; z, c_0, c).$$

Our negative result. Below we give our negative result showing limitations of multi-party NIKE in the BSM.

Theorem 5.15 Let $\mathcal{C}_1$ be the class of streaming word-RAMs with memory bounds m_u . Let $\text{NIKE} = \{\text{Gen}_\lambda, \text{Share}_\lambda\}$ be a $\mathcal{C}_1$ - n_p -NIKE for any constant $n_p \geq 2$ in the BSM with URS length n and satisfying 0-restricted extendability and $((\mu_i)_{i \in [n_p]}, \lfloor m_a/m_u \rfloor)$ -local key independence for any $m_a \geq m_u$, where the associated combining and extracting algorithm families are $\{\text{Combine}_\lambda, \text{Extract}_\lambda\}$.³ Then

$$H(\text{key} \mid (\text{pk}_i)_{i \in [n_p]}) \leq m_u^{n_p} / m_a^{n_p-1} + 2 \sum_{i=2}^{n_p} (m_u/m_a)^{n_p-i} \mu_i,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [n_p]$, and $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$.

Proof. Let $\text{localkey}_{[i]} = \text{Combine}_\lambda(\text{localkey}_{[i-1]}, \text{pk}_i)$ for all $i > 1$, where $\text{localkey}_{[1]} = \text{sk}_1$. Let $M_i = (\text{pk}_i, \text{sk}_i)$ and $\text{localkey}'_{[i]} = \text{Combine}_\lambda(\text{sk}_i, (\text{pk}_j)_{j \geq i})$ for all $i \in [n_p]$. Let $t = \lfloor m_a/m_u \rfloor$, sample $M'_i = (\text{pk}'_i, \text{sk}'_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for all $i \in [t]$, and set $M_E = (M'_i)_{i \in [t]}$.

Fix any $i > 1$. We apply Theorem 5.14 with

$$x = \text{localkey}_{[i-1]}, y = M_i, z = M_E, c_0 = (\text{pk}_j)_{j \in [i-1]}, c = \text{pk}_i, s = \text{localkey}_{[i]}, s' = \text{localkey}'_{[i]}.$$

By 0-restricted extendability and determinism of Combine_λ , we have

$$\begin{aligned} H(\text{localkey}_{[i]} \mid \text{localkey}_{[i-1]}, (\text{pk}_j)_{j \in [i-1]}, \text{pk}_i) &= 0, \\ H(\text{localkey}'_{[i]} \mid M_i, (\text{pk}_j)_{j \in [i-1]}, \text{pk}_i) &= 0, \\ \Pr[\text{localkey}_{[i]} \neq \text{localkey}'_{[i]}] &= 0. \end{aligned}$$

Moreover, since $H(\text{pk}_i \mid M_i) = 0$, we have

$$\begin{aligned} &I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}) \\ &= I(\text{localkey}_{[i-1]}; \text{pk}_i, M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}) \\ &= I(\text{localkey}_{[i-1]}; \text{pk}_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}) + I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}, \text{pk}_i) \\ &\geq I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}, \text{pk}_i). \end{aligned}$$

Therefore, by Theorem 5.14 and local key independence, we have

$$H(\text{localkey}_{[i]}) \leq I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}) + \mu_i.$$

Next, by the second condition in Theorem 2.4, we have

$$I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}) \leq I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}, \text{urs}) + \mu_i.$$

Conditioned on urs and $(\text{pk}_j)_{j \in [i-1]}$, the variables $M_i, M'_1, \dots, M'_t$ are independent copies generated in the same way. Hence, by Theorem 5.13, we have

$$I(\text{localkey}_{[i-1]}; M_i \mid M_E, (\text{pk}_j)_{j \in [i-1]}, \text{urs}) \leq \frac{H(\text{localkey}_{[i-1]} \mid \text{urs}, (\text{pk}_j)_{j \in [i-1]})}{t+1} \leq \frac{H(\text{localkey}_{[i-1]})}{t+1}.$$

³We use $t = \lfloor m_a/m_u \rfloor$. This only affects constant factors; for readability one may assume that m_a/m_u is an integer.

Since $t = \lfloor m_a/m_u \rfloor$, we have $1/(t+1) \leq m_u/m_a$. Combining the above inequalities gives

$$H(\text{localkey}_{[i]}) \leq (m_u/m_a)H(\text{localkey}_{[i-1]}) + 2\mu_i.$$

We now iterate this recurrence. Since $H(\text{localkey}_{[1]}) \leq H(M_1) \leq m_u$, we have

$$H(\text{localkey}_{[2]}) \leq (m_u/m_a)H(\text{localkey}_{[1]}) + 2\mu_2 \leq m_u^2/m_a + 2\mu_2.$$

Assume that, for some $i \geq 3$,

$$H(\text{localkey}_{[i-1]}) \leq m_u^{i-1}/m_a^{i-2} + 2 \sum_{\ell=2}^{i-1} (m_u/m_a)^{i-1-\ell} \mu_\ell.$$

Then we have

$$H(\text{localkey}_{[i]}) \leq (m_u/m_a)H(\text{localkey}_{[i-1]}) + 2\mu_i \leq m_u^i/m_a^{i-1} + 2 \sum_{\ell=2}^i (m_u/m_a)^{i-\ell} \mu_\ell.$$

Thus,

$$H(\text{localkey}_{[n_p]}) \leq m_u^{n_p}/m_a^{n_p-1} + 2 \sum_{i=2}^{n_p} (m_u/m_a)^{n_p-i} \mu_i.$$

Finally, conditioning on the public keys,

$$\begin{aligned} H(\text{key} \mid (\text{pk}_i)_{i \in [n_p]}) &= H(\text{Extract}_\lambda(\text{localkey}_{[n_p]}, \text{pk}_{n_p}) \mid (\text{pk}_i)_{i \in [n_p]}) \leq H(\text{localkey}_{[n_p]} \mid (\text{pk}_i)_{i \in [n_p]}) \\ &\leq H(\text{localkey}_{[n_p]}) \leq m_u^{n_p}/m_a^{n_p-1} + 2 \sum_{i=2}^{n_p} (m_u/m_a)^{n_p-i} \mu_i. \end{aligned}$$

This completes the proof of Theorem 5.15. $\square$

Optimality of BSMKE. Let n_p , n , λ , and q be as in Section 5. By Section A.4, BSMKE satisfies 0-extendability, while aborts are covered by the $2/\lambda$ -correctness bound in Theorem 5.9. Thus, it remains to verify local key independence. The next lemma gives the required bounds and shows that our instantiation matches the lower bound up to constant factors.

Lemma 5.16 *For every $t \in \mathbb{N}$, BSMKE satisfies $((\mu_i)_{i \in [n_p]}, t)$ -local key independence in the BSM with*

$$\mu_i = O(t \cdot q^{i+1}/n^i + q^i/n^{i-1} + \log n) \text{ for all } i \in [n_p].$$

Proof. For notational simplicity, let $\mathcal{S}_i$ be the index set stored by the i -th user, and let $\mathcal{S}_{[i]} = \mathcal{S}_1 \cap \dots \cap \mathcal{S}_i$, so that $\text{localkey}_{[i]} = \text{urs}_{\mathcal{S}_{[i]}}$. We suppress the compact public hash descriptions defining these sets; their size is $O(\log n)$ and is absorbed into the $O(\log n)$ term.

Fix $i \in [n_p]$, and let $\mathcal{S}'_j$ be the index set stored in sk'_j . The public keys reveal the length of the variable-length string $\text{urs}_{\mathcal{S}_{[i]}}$, which costs at most $O(\log n)$ bits. Once the public keys fix the index sets, the only correlation between $\text{urs}_{\mathcal{S}_{[i]}}$ and $\text{urs}_{\mathcal{S}'_1 \cup \dots \cup \mathcal{S}'_t}$ comes from their overlapping positions. Hence

$$\begin{aligned} &I(\text{localkey}_{[i]}; (\text{pk}_j)_{j \in [i]}, (\text{pk}'_j, \text{sk}'_j)_{j \in [t]}) \\ &= I(\text{urs}_{\mathcal{S}_{[i]}}; (\text{pk}_j)_{j \in [i]}, (\text{pk}'_j)_{j \in [t]}) + I(\text{urs}_{\mathcal{S}_{[i]}}; (\text{sk}'_j)_{j \in [t]} \mid (\text{pk}_j)_{j \in [i]}, (\text{pk}'_j)_{j \in [t]}) \\ &\leq O(\log n) + I(\text{urs}_{\mathcal{S}_{[i]}}; \text{urs}_{\mathcal{S}'_1 \cup \dots \cup \mathcal{S}'_t} \mid (\text{pk}_j)_{j \in [i]}, (\text{pk}'_j)_{j \in [t]}) \\ &\leq O(\log n) + \mathbb{E}[|\mathcal{S}_{[i]} \cap (\mathcal{S}'_1 \cup \dots \cup \mathcal{S}'_t)|] \\ &\leq O(\log n) + \sum_{j=1}^t \mathbb{E}[|\mathcal{S}_{[i]} \cap \mathcal{S}'_j|] \\ &= O(\log n) + t \sum_{a \in GF(n)} \Pr[a \in \mathcal{S}_{[i]} \cap \mathcal{S}'_1] \\ &= O(\log n) + t \cdot n(q/n)^{i+1} \\ &= O(t \cdot q^{i+1}/n^i + \log n). \end{aligned}$$

This proves the first condition.

It remains to verify the second condition. For $i > 1$, conditioned on $\mathbf{urs}$ and $(\mathbf{pk}_j)_{j \in [i-1]}$, the value $\mathbf{localkey}_{[i-1]} = \mathbf{urs}_{\mathcal{S}_{[i-1]}}$ is fixed. Therefore,

$$I(\mathbf{localkey}_{[i-1]}; (\mathbf{pk}_i, \mathbf{sk}_i) \mid (\mathbf{pk}_j)_{j \in [i-1]}, (\mathbf{pk}'_j, \mathbf{sk}'_j)_{j \in [t]}, \mathbf{urs}) = 0.$$

Without conditioning on $\mathbf{urs}$, the new key pair $(\mathbf{pk}_i, \mathbf{sk}_i)$ can reveal information about $\mathbf{localkey}_{[i-1]}$ only through the overlap $\mathcal{S}_{[i-1]} \cap \mathcal{S}_i = \mathcal{S}_{[i]}$. Hence

$$I(\mathbf{localkey}_{[i-1]}; (\mathbf{pk}_i, \mathbf{sk}_i) \mid (\mathbf{pk}_j)_{j \in [i-1]}, (\mathbf{pk}'_j, \mathbf{sk}'_j)_{j \in [t]}) \leq \mathbb{E}[|\mathcal{S}_{[i]}|] = \sum_{a \in GF(n)} \Pr[a \in \mathcal{S}_{[i]}] = n(q/n)^i = q^i/n^{i-1}.$$

Thus we have

$$\begin{aligned} & I(\mathbf{localkey}_{[i-1]}; (\mathbf{pk}_i, \mathbf{sk}_i) \mid (\mathbf{pk}_j)_{j \in [i-1]}, (\mathbf{pk}'_j, \mathbf{sk}'_j)_{j \in [t]}) \\ & \leq I(\mathbf{localkey}_{[i-1]}; (\mathbf{pk}_i, \mathbf{sk}_i) \mid (\mathbf{pk}_j)_{j \in [i-1]}, (\mathbf{pk}'_j, \mathbf{sk}'_j)_{j \in [t]}, \mathbf{urs}) + q^i/n^{i-1}. \end{aligned}$$

Since $\mu_i = O(t \cdot q^{i+1}/n^i + q^i/n^{i-1} + \log n)$, both conditions in Theorem 2.4 are satisfied. $\square$

Remark. For the usual parameter choice $n = \Theta(\mathbf{m}_a)$, $\mathbf{m}_u = \Theta(q)$, and $t = \lfloor \mathbf{m}_a/\mathbf{m}_u \rfloor$, Theorem 5.16 gives

$$t \cdot q^{i+1}/n^i = O((\mathbf{m}_a/\mathbf{m}_u) \cdot \mathbf{m}_u^{i+1}/\mathbf{m}_a^i) = O(\mathbf{m}_u^i/\mathbf{m}_a^{i-1}) \text{ and } q^i/n^{i-1} = O(\mathbf{m}_u^i/\mathbf{m}_a^{i-1}).$$

Hence, we have $\mu_i = O(\mathbf{m}_u^i/\mathbf{m}_a^{i-1} + \log n)$. Plugging this into Theorem 5.15 yields

$$H(\mathbf{key} \mid (\mathbf{pk}_i)_{i \in [n_p]}) \leq O(\mathbf{m}_u^{n_p}/\mathbf{m}_a^{n_p-1} + \log n).$$

When $\log n = o(\lambda)$, any scheme in this class with $H(\mathbf{key} \mid (\mathbf{pk}_i)_{i \in [n_p]}) = \Omega(\lambda)$ must satisfy

$$\mathbf{m}_u = \Omega\left(\sqrt[n_p]{\lambda \cdot \mathbf{m}_a^{n_p-1}}\right).$$

Thus, the BSM construction is optimal up to constant factors within this class.

Acknowledgements

Part of this work was supported by the National Natural Science Foundation of China under Grant Numbers 62472073 and 62272091.

References

- [1] Backurs, A., Tzamos, C.: Improving viterbi is hard: better runtimes imply faster clique algorithms. In: Proceedings of the 34th International Conference on Machine Learning - Volume 70. p. 311321. ICML'17, JMLR.org (2017) (Cited on page 2.)
- [2] Diffie, W., Hellman, M.E.: New directions in cryptography. IEEE Trans. Inf. Theory 22(6), 644–654 (1976) (Cited on page 1.)
- [3] Goldreich, O., Levin, L.A.: A hard-core predicate for all one-way functions. In: STOC. pp. 25–32. ACM (1989) (Cited on page 7.)
- [4] Guo, S., Kamath, P., Rosen, A., Sotiraki, K.: Limits on the efficiency of (ring) LWE-based non-interactive key exchange. J. Cryptol. 35(1), 1 (2022) (Cited on page 1.)
- [5] Håstad, J.: One-way permutations in NC0. Inf. Process. Lett. 26(3), 153–155 (1987) (Cited on page 2.)
- [6] Hofheinz, D.: Fully secure constrained pseudorandom functions using random oracles. Cryptology ePrint Archive, Paper 2014/372 (2014), <https://eprint.iacr.org/2014/372> (Cited on page 4, 7, 32.)

- [7] Joux, A.: A one-round protocol for tripartite Diffie-Hellman. *J. Cryptol.* 17(4), 263–276 (2004)
(Cited on page 1, 4.)
- [8] Maurer, U.M.: Conditionally-perfect secrecy and a provably-secure randomized cipher. *J. Cryptol.* 5(1), 53–66 (1992) (Cited on page 2.)
- [9] Maurer, U.M.: Secret key agreement by public discussion from common information. *IEEE Trans. Inf. Theory* 39(3), 733–742 (1993), <https://doi.org/10.1109/18.256484> (Cited on page 8, 27.)
- [10] Merkle, R.C.: Secure communications over insecure channels. *Commun. ACM* 21(4), 294–299 (1978)
(Cited on page 2.)
- [11] Razborov, A.A.: Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical notes of the Academy of Sciences of the USSR* 41(4) (Apr 1987)
(Cited on page 2, 10.)

A Fine-Grained Multi-Party IB-NIKE and BPRF

In this section, we propose multi-party IB-NIKE schemes in the fine-grained setting. We first give the definitions of fine-grained multi-party IB-NIKE and BPRF, and then give a generic construction of multi-party IB-NIKE based on BPRF. Next we present a generic construction of BPRF from multi-party NIKE with an additional property we refer to as extendability, which is satisfied by all our multi-party NIKE schemes. Throughout this section, we focus on IB-NIKE for a constant number of parties with a larger, constant-size identity space, and BPRF with constant-size input space.

A.1 Definitions

We now give the definitions of IB-NIKE and BPRF in the fine-grained setting.

Definition A.1 (Identity-based non-interactive key exchange). A $\mathcal{C}_1$ - n_p -identity-based non-interactive key exchange (IB-NIKE) consists of three algorithm families $\text{IB-NIKE} = \{\text{IBSetup}_\lambda, \text{IBExtract}_\lambda, \text{IBShare}_\lambda\} \in \mathcal{C}_1$ such that:

- IBSetup_λ on input $\text{urs} \in \{0, 1\}^n$ for some $n = n(\lambda)$ outputs a master secret key ibmsk .
- IBExtract_λ is a deterministic algorithm that on input a master secret key ibmsk and an identity $\text{id} \in \mathcal{ID}_\lambda$, where $\mathcal{ID}_\lambda$ is the identity space with *constant size*, outputs a secret key ibsk_{id} for id .
- IBShare_λ is a deterministic algorithm that on input a list $\mathcal{I} \subseteq \mathcal{ID}_\lambda$ consisting of n_p identities and a secret key ibsk_{id} for $\text{id} \in \mathcal{I}$ outputs a shared key $\text{ibshk} \in \mathcal{K}_\lambda$ for $\mathcal{I}$, where $\mathcal{K}_\lambda$ is the shared key space. We assume that the identities in $\mathcal{I}$ are always lexicographically ordered.

ϵ -correctness is satisfied if for all $\lambda \in \mathbb{N}$, all lists of n_p distinct identities $\mathcal{I} \subseteq \mathcal{ID}_\lambda$, all $\text{id}_i, \text{id}_j \in \mathcal{I}$, we have

$$\Pr[\text{IBShare}_\lambda(\text{ibsk}_{\text{id}_i}, \mathcal{I}) = \text{IBShare}_\lambda(\text{ibsk}_{\text{id}_j}, \mathcal{I})] \geq 1 - \epsilon,$$

where $\text{ibsk}_{\text{id}_i} = \text{IBExtract}_\lambda(\text{ibmsk}, \text{id}_i)$ and $\text{ibsk}_{\text{id}_j} = \text{IBExtract}_\lambda(\text{ibmsk}, \text{id}_j)$ for $\text{urs} \xleftarrow{\$} \{0, 1\}^n$ and $\text{ibmsk} \xleftarrow{\$} \text{IBSetup}_\lambda(\text{urs})$.

$\mathcal{C}_2$ - ϵ -security is satisfied if for any adversary $\mathcal{A} = \{\text{adv}_\lambda\} \in \mathcal{C}_2$, all sufficiently large $\lambda \in \mathbb{N}$, and any list of n_p distinct identities $\mathcal{I}^* \subseteq \mathcal{ID}_\lambda$, we have

$$\begin{aligned} & \left| \Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, \text{ibshk}, (\text{ibsk}_{\text{id}})_{\text{id} \in \mathcal{ID}_\lambda \setminus \mathcal{I}^*})] \right. \\ & \quad \left. - \Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, \text{ibshk}', (\text{ibsk}_{\text{id}})_{\text{id} \in \mathcal{ID}_\lambda \setminus \mathcal{I}^*})] \right| \leq \epsilon, \end{aligned}$$

where $\text{ibmsk} \xleftarrow{\$} \text{IBSetup}_\lambda(\text{urs})$, $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\text{ibshk} = \text{IBShare}_\lambda(\text{ibsk}_{\text{id}^*}, \mathcal{I}^*)$, $\text{ibshk}' \xleftarrow{\$} \mathcal{K}_\lambda$, $\text{id}^* \in \mathcal{I}^*$, and $\text{ibsk}_{\text{id}} = \text{IBExtract}_\lambda(\text{ibmsk}, \text{id})$ for any $\text{id} \in \mathcal{ID}_\lambda$.

(m_a, ϵ, θ) -security in the BSM is satisfied if for any function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{m_a}$ and any list of n_p distinct identities $\mathcal{I}^* \subseteq \mathcal{ID}_\lambda$, we have

$$\exists E : \Pr[E] \geq 1 - \epsilon \text{ and } I(\text{ibshk}; (f(\text{urs}), (\text{ibsk}_{\text{id}})_{\text{id} \in \mathcal{ID}_\lambda \setminus \mathcal{I}^*}) \mid E) \leq \theta,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\text{ibsk}_{\text{id}} = \text{IBExtract}_\lambda(\text{ibmsk}, \text{id})$ for $\text{ibmsk} \xleftarrow{\$} \text{IBSetup}_\lambda(\text{urs})$, and $\text{ibshk} = \text{IBShare}_\lambda(\text{ibsk}_{\text{id}}, \mathcal{I}^*)$ for some $\text{id} \in \mathcal{I}^*$.

We now define two types of sets

$$\mathcal{S}_{\hat{x}, \ell_{\text{inp}}} = \{x = (x_i)_{i \in [\ell_{\text{inp}}]} \in \{0, 1\}^{\ell_{\text{inp}}} \mid \forall i : x_i = \hat{x}_i \vee \hat{x}_i = \perp\}$$

for any $\hat{x} = (\hat{x}_i)_{i \in [\ell_{\text{inp}}]} \in (\{0, 1, \perp\})^{\ell_{\text{inp}}}$, and

$$\mathcal{T}_{x, \ell_{\text{inp}}} = \{\hat{x} \in \{0, 1, \perp\}^{\ell_{\text{inp}}} \mid x \notin \mathcal{S}_{\hat{x}, \ell_{\text{inp}}}\}$$

for any $x \in \{0, 1\}^{\ell_{\text{inp}}}$, and give the definition of BPRF and its restricted version, both of which can be converted into multi-party IB-NIKES, as shown below. Here we note that BPRF in the fine-grained setting is of independent interest, since it also generically implies broadcast encryption with constant users in the fine-grained setting, using the technique in [?].

Definition A.2 (Bit-fixing pseudorandom functions). A $\mathcal{C}_1$ - ℓ_{inp} -bit-fixing pseudorandom function (BPRF) consists of four algorithm families $\text{BPRF} = \{\text{Setup}_\lambda, \text{Eval}_\lambda, \text{Constrain}_\lambda, \text{CEval}_\lambda\} \in \mathcal{C}_1$ such that

- Setup_λ on input $\text{urs} \in \{0, 1\}^n$ for some $n = n(\lambda)$ outputs a master secret key msk .
- Eval_λ is a deterministic algorithm that on input a master secret key msk and a preimage $x \in \{0, 1\}^{\ell_{\text{inp}}}$ outputs an image $y \in \mathcal{Y}$.
- Constrain_λ is a deterministic algorithm that on input a master secret key msk and a string $\hat{x}$ outputs a constrained key $\text{bfk}_{\hat{x}} \in \mathcal{CK}_\lambda$, where $\mathcal{CK}_\lambda$ is the constrained key space.
- CEval_λ is a deterministic algorithm that on input a constrained key $\text{bfk}_{\hat{x}}$ and a preimage $x \in \mathcal{S}_{\hat{x}, \ell_{\text{inp}}}$ outputs an image $y \in \mathcal{Y}$.

ϵ -correctness is satisfied if for all $\hat{x} \in (\{0, 1, \perp\})^{\ell_{\text{inp}}}$ and all x such that $x \in \mathcal{S}_{\hat{x}, \ell_{\text{inp}}}$, we have

$$\Pr[\text{CEval}_\lambda(\text{bfk}_{\hat{x}}, x) = \text{Eval}_\lambda(\text{msk}, x)] \geq 1 - \epsilon,$$

where $\text{bfk}_{\hat{x}} = \text{Constrain}_\lambda(\text{msk}, \hat{x})$ for $\text{urs} \xleftarrow{\$} \{0, 1\}^n$ and $\text{msk} \xleftarrow{\$} \text{Setup}_\lambda(\text{urs})$.

$\mathcal{C}_2$ - ϵ -security is satisfied if for any adversary $\mathcal{A} = \{\text{adv}_\lambda\} \in \mathcal{C}_2$, all sufficiently large $\lambda \in \mathbb{N}$, and any $x \in \{0, 1\}^{\ell_{\text{inp}}}$, we have

$$|\Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, y, \{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, \ell_{\text{inp}}}})] - \Pr[1 \xleftarrow{\$} \text{adv}_\lambda(\text{urs}, y', \{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, \ell_{\text{inp}}}})]| \leq \epsilon,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\text{msk} \xleftarrow{\$} \text{Setup}_\lambda(\text{urs})$, $y \xleftarrow{\$} \text{Eval}_\lambda(\text{msk}, x)$, $y' \xleftarrow{\$} \mathcal{Y}$, and $\text{bfk}_{\hat{x}} = \text{Constrain}_\lambda(\text{msk}, \hat{x})$ for all $\hat{x} \in \mathcal{T}_{x, \ell_{\text{inp}}}$.

(m_a, ϵ, θ) -security in the BSM is satisfied if for any function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{m_a}$ and any $x \in \{0, 1\}^{\ell_{\text{inp}}}$, we have

$$\exists E : \Pr[E] \geq 1 - \epsilon \text{ and } I(y; (f(\text{urs}), \{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, \ell_{\text{inp}}}}) \mid E) \leq \theta,$$

where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\text{msk} \xleftarrow{\$} \text{Setup}_\lambda(\text{urs})$, $\text{bfk}_{\hat{x}} = \text{Constrain}_\lambda(\text{msk}, \hat{x})$ for all $\hat{x} \in \mathcal{T}_{x, \ell_{\text{inp}}}$, and $y = \text{Eval}_\lambda(\text{msk}, x)$.

Definition A.3 (Restricted bit-fixing pseudorandom functions). A $\mathcal{C}_1$ - ℓ_{inp} -restricted bit-fixing pseudorandom function (restricted BPRF) is defined in exactly the same way as $\mathcal{C}_1$ - ℓ_{inp} -BPRF except that we require that $\hat{x} \in \{\perp\}^{t_1-1} \times \{0, 1\}^{t_2-t_1+1} \times \{\perp\}^{\ell_{\text{inp}}-t_2}$ for all $t_1, t_2 \in [\ell_{\text{inp}}]$.

A.2 Constructions of Multi-Party IB-NIKE

In this section, we give our constructions of multi-party IB-NIKE in the bounded-parallel-time model, bounded-time model, and BSM, respectively. These constructions are similar to those in [6], with the distinction that the underlying BPRFs can be restricted BPRFs and are built in the fine-grained setting.

Construction in the bounded-parallel-time model. Let $\text{NCBPRF} = \{\text{Setup}_\lambda, \text{Eval}_\lambda, \text{Constrain}_\lambda, \text{CEval}_\lambda\}$ be an $\text{AC}^0[2]$ - n_p -restricted BPRF with URS length n and image space $\mathcal{Y}$ for some constant n_p , and some $n = n(\lambda)$. Let m_p be any constant such that n_p/m_p is an integer. Our $\text{NCIB-NIKE} = \{\text{IBSetup}_\lambda, \text{IBExtract}_\lambda, \text{IBShare}_\lambda\}$ with identity space $\{0, 1\}^{n_p/m_p}$ and URS length n is defined as in Figure 15.

Theorem A.4 If NCBPRF satisfies ϵ_1 -correctness and NC^1 - ϵ_2 -security, then NCIB-NIKE in Figure 15 is an $\text{AC}^0[2]$ - m_p -IB-NIKE with $2 \cdot \epsilon_1$ -correctness and NC^1 - ϵ_2 -security.

Proof. Complexity. First note that $\{\text{IBSetup}_\lambda, \text{IBExtract}_\lambda, \text{IBShare}_\lambda\}$ are computable in $\text{AC}^0[2]$ since they run Constrain_λ and CEval_λ for a constant number of times.

Correctness. For any list of m_p distinct identities $\mathcal{I}^* \subseteq \{0, 1\}^{n_p/m_p}$ and any $i, j \in [m_p]$, by the ϵ_1 -correctness of NCBPRF , we have

$$\Pr[\text{CEval}_\lambda(\text{bfk}_i, \mathcal{I}^*) = \text{Eval}_\lambda(\text{ibmsk}, \mathcal{I}^*)] \geq 1 - \epsilon_1$$

$$\text{and } \Pr[\text{CEval}_\lambda(\text{bfk}_j, \mathcal{I}^*) = \text{Eval}_\lambda(\text{ibmsk}, \mathcal{I}^*)] \geq 1 - \epsilon_1,$$

where $\text{ibmsk} \xleftarrow{\$} \text{Setup}_\lambda(\text{urs})$ for $\text{urs} \xleftarrow{\$} \{0, 1\}^n$ and $\text{bfk}_i = \text{Constrain}_\lambda(\text{ibmsk}, \hat{x}_i)$ for $\hat{x}_i \in \{0, 1, \perp\}^{n_p}$ and any $i \in [m_p]$. Therefore by a union bound, we have

$$\Pr[\text{CEval}_\lambda(\text{bfk}_i, \mathcal{I}^*) = \text{CEval}_\lambda(\text{bfk}_j, \mathcal{I}^*)] \geq 1 - 2\epsilon_1,$$

$$\text{i.e., } \Pr[\text{IBShare}_\lambda(\text{ibsk}_{\text{id}_i}, \mathcal{I}^*) = \text{IBShare}_\lambda(\text{ibsk}_{\text{id}_j}, \mathcal{I}^*)] \geq 1 - 2\epsilon_1,$$

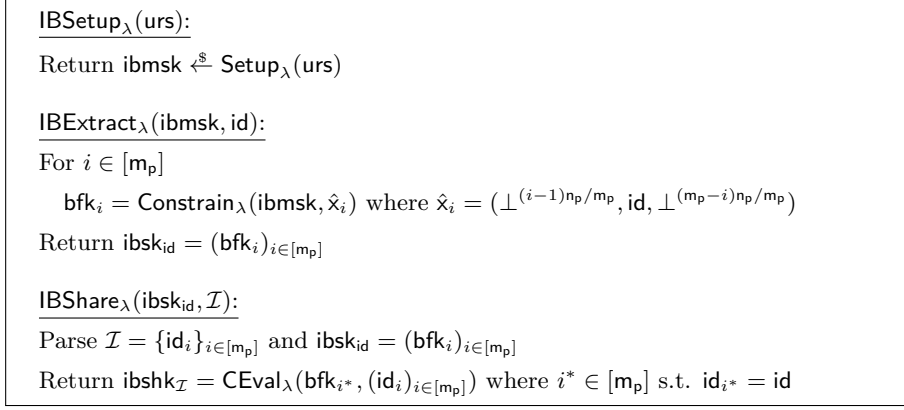


Figure 15: Construction of NCIB-NIKE = $\{\text{IBSetup}_{\lambda}, \text{IBExtract}_{\lambda}, \text{IBShare}_{\lambda}\}$.

where $\text{ibsk}_{\text{id}_i} = \text{IBExtract}_{\lambda}(\text{ibmsk}, \text{id}_i)$ for $\text{ibmsk} \xleftarrow{\$} \text{IBSetup}_{\lambda}(\text{urs})$ and any $i \in [m_p]$.

Security. Let λ be the security parameter and $\mathcal{A} = \{\text{adv}_{\lambda}\} \in \text{NC}^1$ be any adversary against the NC^1 -security of NCIB-NIKE. For any $x \in \{0, 1\}^{n_p}$, we construct $\mathcal{B} = \{\text{advb}_{\lambda}\} \in \text{NC}^1$ against the security of NCBPRF as follows.

On receiving $(y, \{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, n_p}})$ where $\text{bfk}_{\hat{x}} = \text{Constrain}_{\lambda}(\text{msk}, \hat{x})$ for all $\hat{x} \in \mathcal{T}_{x, n_p}$ and $y \xleftarrow{\$} \text{Eval}_{\lambda}(\text{msk}, x)$ or $y \xleftarrow{\$} \mathcal{Y}$ where $\text{msk} \xleftarrow{\$} \text{Setup}_{\lambda}(\text{urs})$ for $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, advb_{λ} returns $\text{ibshk} = y$ and $\{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, n_p}}$ to adv_{λ} . On receiving $b \in \{0, 1\}$ from adv_{λ} , advb_{λ} forwards b to its challenger.

First we note that all operations in advb_{λ} are in NC^1 , since advb_{λ} does not execute any additional operations other than running adv_{λ} . Moreover, when $y \xleftarrow{\$} \text{Eval}_{\lambda}(\text{msk}, x)$ (respectively, $y \xleftarrow{\$} \mathcal{Y}$) for any $x \in \{0, 1\}^{n_p}$ where $\text{msk} \xleftarrow{\$} \text{Setup}_{\lambda}(\text{urs})$ for $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, the view of adv_{λ} is identical to its view in the security game for NCIB-NIKE when ibshk is honestly generated (respectively, generated at random). Hence, the advantage of advb_{λ} in breaking the security of NCBPRF is the same as the advantage of adv_{λ} in breaking the security of NCIB-NIKE, completing the proof of Theorem A.4. $\square$

Construction in the bounded-time model and BSM. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be the set of all word-RAM families such that for each $\mathcal{F}_1 = \{f_{\lambda}^1\} \in \mathcal{C}_1$ and $\mathcal{F}_2 = \{f_{\lambda}^2\} \in \mathcal{C}_2$ and each $\lambda \in \mathbb{N}$, f_{λ}^1 runs in time $\tilde{O}(T_1(\lambda))$ and f_{λ}^2 runs in time $\tilde{O}(T_2(\lambda))$. Let m_p be any constant. Our construction in the bounded-time model TIB-NIKE with URS length n is defined in exactly the same way as NCIB-NIKE (see Figure 15) except that the underlying NCBPRF is replaced by a $\mathcal{C}_1$ - n_p -BPRF with URS length n and image length ℓ for some constant n_p , some ℓ , and some $n = n(\lambda)$. Notice that for our instantiation of $\mathcal{C}_1$ - n_p -BPRF given later in Section A.3, we have $n = 0$, i.e., the IB-NIKE is in the plain model.

Moreover, let $\mathcal{C}'_1$ be the class of streaming word-RAM families with memory bounds m_u . Let m_p' be any constant. Our construction in the BSM BSIB-NIKE with URS length n' is defined in exactly the same way as NCIB-NIKE (see Figure 15) except that the underlying NCBPRF is replaced by a $\mathcal{C}'_1$ - n_p -BPRF with URS length n' and image length ℓ' for some constant n_p , some ℓ' , and some $n' = n'(\lambda)$. We have the following theorems.

Theorem A.5 *If TBPRF is a $\mathcal{C}_1$ - n_p -BPRF with ϵ_1 -correctness and $\mathcal{C}_2$ - ϵ_2 -security, then TIB-NIKE is a $\mathcal{C}_1$ - m_p -IB-NIKE with $2 \cdot \epsilon_1$ -correctness and $\mathcal{C}_2$ - ϵ_2 -security.*

Theorem A.6 *If BSIBPRF is a $\mathcal{C}'_1$ - n_p -BPRF with ϵ'_1 -correctness and $(m_a, \epsilon', \epsilon'_2)$ -security in the BSM, then BSIB-NIKE is a $\mathcal{C}'_1$ - m_p' -IB-NIKE with $2 \cdot \epsilon'_1$ -correctness and $(m_a, \epsilon', \epsilon'_2)$ -security in the BSM.*

The proofs of Theorems A.5 and A.6 are very similar to the proof of Theorem A.4 and thus we omit the details.

A.3 BPRF from Multi-Party NIKE with Extendability

In this section, we instantiate (restricted) BPRF based on our multi-party NIKE with (restricted) extendability, and then show that all our multi-party NIKE schemes satisfy extendability.

Restricted BPRF in the bounded-parallel-time model. Let $\text{NCNIKE} = \{\text{Gen}_\lambda, \text{Share}_\lambda\}$ be an $\text{AC}^0[2]$ - n_p -NIKE with shared key space $\mathcal{K}_\lambda$ and URS length n for some constant n_p , and some $n = n(\lambda)$ satisfying restricted ϵ_1 -extendability with associated algorithm families $\{\text{Combine}_\lambda, \text{Extract}_\lambda\}$ and NC^1 - ϵ_2 -security. Our $\text{NCBPRF} = \{\text{Setup}_\lambda, \text{Eval}_\lambda, \text{Constrain}_\lambda, \text{CEval}_\lambda\}$ is defined as in Figure 16.

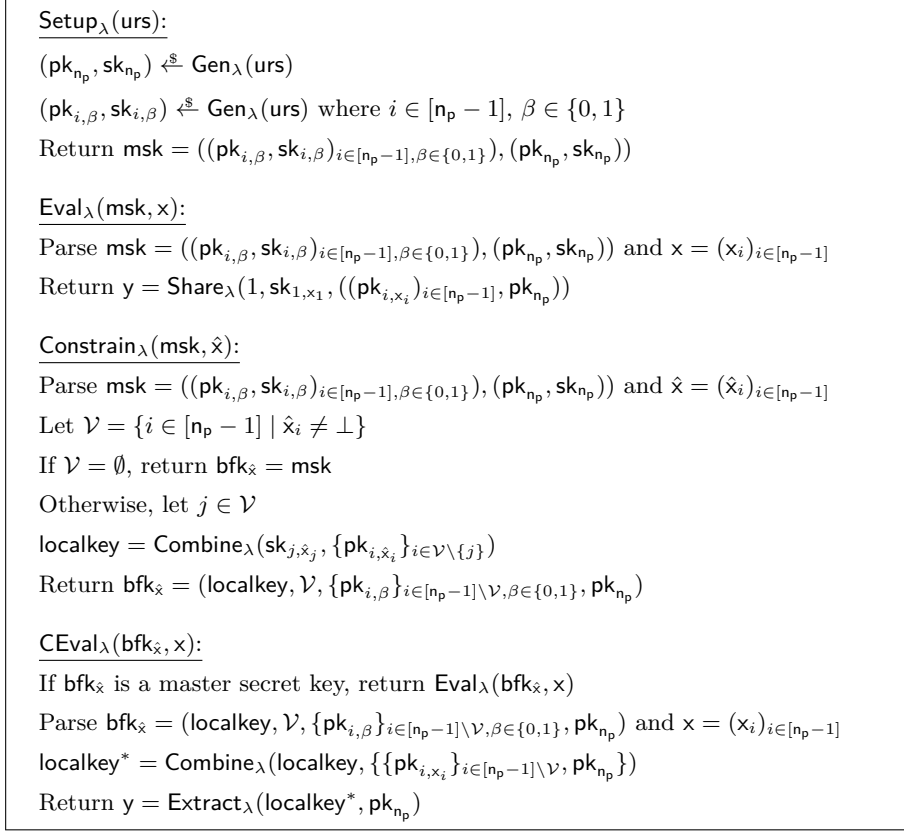


Figure 16: Construction of $\text{NCBPRF} = \{\text{Setup}_\lambda, \text{Eval}_\lambda, \text{Constrain}_\lambda, \text{CEval}_\lambda\}$.

Theorem A.7 *NCBPRF is an $\text{AC}^0[2]$ -($n_p - 1$)-restricted BPRF with ϵ_1 -correctness and NC^1 - ϵ_2 -security.*

Proof. Complexity. First we note that the constructions are computable in $\text{AC}^0[2]$ since they only run Gen_λ and Share_λ for a constant number of times.

Correctness. Let $t_1, t_2 \in [n_p - 1]$ and set $\mathcal{V} = [t_1, t_2]$. Fix $\hat{x} = (\hat{x}_i)_{i \in [n_p - 1]} \in \{\perp\}^{t_1 - 1} \times \{0, 1\}^{t_2 - t_1 + 1} \times \{\perp\}^{n_p - 1 - t_2}$ and $x \in \mathcal{S}_{\hat{x}, n_p - 1}$. Consider the experiment where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $\text{msk} \xleftarrow{\$} \text{Setup}_\lambda(\text{urs})$, and $\text{bfk}_{\hat{x}} = \text{Constrain}_\lambda(\text{msk}, \hat{x})$. Let E be the event guaranteed by the ϵ_1 -restricted extendability of NCNIKE for the selected key pairs $((\text{pk}_{i,x_i}, \text{sk}_{i,x_i})_{i \in [n_p - 1]}, (\text{pk}_{n_p}, \text{sk}_{n_p}))$. For any $i' \in \mathcal{V}$, let

$$\text{localkey} = \text{Combine}_\lambda(\text{sk}_{i',x_{i'}}, \{\text{pk}_{i,x_i}\}_{i \in \mathcal{V} \setminus \{i'\}})$$

and

$$\text{localkey}^* = \text{Combine}_\lambda(\text{localkey}, \{\{\text{pk}_{i,x_i}\}_{i \in [n_p - 1] \setminus \mathcal{V}}, \text{pk}_{n_p}\}).$$

Then $\Pr[E] \geq 1 - \epsilon_1$, and conditioned on E ,

$$\text{localkey}^* = \text{Combine}_\lambda(\text{sk}_{1,x_1}, \{\{\text{pk}_{i,x_i}\}_{1 < i \leq n_p - 1}, \text{pk}_{n_p}\})$$

Thus,

$$\begin{aligned} & \Pr[\text{CEval}_\lambda(\text{bfk}_{\hat{x}}, x) = \text{Eval}_\lambda(\text{msk}, x)] \\ & \geq \Pr[\text{CEval}_\lambda(\text{bfk}_{\hat{x}}, x) = \text{Eval}_\lambda(\text{msk}, x) \mid E] \Pr[E] \\ & = \Pr[\text{Extract}_\lambda(\text{localkey}^*, \text{pk}_{n_p}) = \text{Share}_\lambda(1, \text{sk}_{1,x_1}, ((\text{pk}_{i,x_i})_{i \in [n_p - 1]}, \text{pk}_{n_p})) \mid E] \Pr[E] \\ & = \Pr[E] \geq 1 - \epsilon_1 \end{aligned}$$

Security. Let λ be the security parameter, and $\mathcal{A} = \{\text{adv}_\lambda\} \in \text{NC}^1$ be any adversary against the NC^1 -security of NCBPRF. For any $x \in \{0, 1\}^{n_p-1}$, we construct an adversary $\mathcal{B} = \{\text{advb}_\lambda\} \in \text{NC}^1$ against the security of NCNIKE as follows.

On receiving $((\text{pk}_i)_{i \in [n_p]}, \text{key})$ where $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for $i \in [n_p]$, and $\text{key} = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$ or $\text{key} \xleftarrow{\$} \mathcal{K}_\lambda$, advb_λ firstly samples $(\text{pk}'_i, \text{sk}'_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for every $i \in [n_p - 1]$.

Then advb_λ sets $\text{pk}_{i,\beta} = \begin{cases} \text{pk}_i & \text{if } x_i = \beta \\ \text{pk}'_i & \text{if } x_i \neq \beta \end{cases}$ and $\text{bfk}_{\hat{x}} = (\text{localkey}, \mathcal{V}, ((\text{pk}_{i,\beta})_{i \in [n_p-1] \setminus \mathcal{V}, \beta \in \{0,1\}}, \text{pk}_{n_p}))$ for any $\hat{x} \in \mathcal{T}_{x, n_p-1}$ where $\text{localkey} = \text{Combine}_\lambda(\text{sk}_{i', \hat{x}_{i'}}, \{\text{pk}_{i, \hat{x}_i}\}_{i \in \mathcal{V} \setminus \{i'\}})$ for some $i' \in \mathcal{V}$ such that $\text{sk}_{i', \hat{x}_{i'}}$ is sampled by advb_λ . Then advb_λ returns $y = \text{key}$ and $\{\text{bfk}_{\hat{x}}\}_{\hat{x} \in \mathcal{T}_{x, n_p-1}}$ to adv_λ . When adv_λ finally returns $b \in \{0, 1\}$, advb_λ forwards b to its challenger.

First we note that all operations in advb_λ are in NC^1 , since advb_λ runs Gen_λ for a constant number of times. Moreover, when $y = \text{Share}_\lambda(1, \text{sk}_1, (\text{pk}_i)_{i \in [n_p]})$ (respectively, $y \xleftarrow{\$} \mathcal{K}_\lambda$) where $(\text{pk}_i, \text{sk}_i) \xleftarrow{\$} \text{Gen}_\lambda(\text{urs})$ for $i \in [n_p]$ and $\text{urs} \xleftarrow{\$} \{0, 1\}^n$, the view of adv_λ is identical to its view in the security game for NCBPRF when y is honestly generated (respectively, generated at random). Hence, the advantage of advb_λ in breaking the security of NCNIKE is the same as the advantage of adv_λ in breaking the security of BPRF, completing the proof of Theorem A.7. $\square$

BPRF in the bounded-time model and BSM. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ represent the set of all word-RAM families such that for each $\mathcal{F}_1 = \{f_\lambda^1\} \in \mathcal{C}_1$ and $\mathcal{F}_2 = \{f_\lambda^2\} \in \mathcal{C}_2$ and each $\lambda \in \mathbb{N}$, f_λ^1 runs in time $\tilde{O}(T_1(\lambda))$ and f_λ^2 runs in time $\tilde{O}(T_2(\lambda))$. Our construction in the bounded-time model TBPRF is defined in exactly the same way as in Figure 16 except that the underlying NCNIKE is replaced by TNIKE*, which is a $\mathcal{C}_1$ - n_p -NIKE with shared key space $\mathcal{K}_\lambda$ and URS length n satisfying ϵ_1 -extendability and $\mathcal{C}_2$ - ϵ_2 -security for some constant n_p , and some $n = n(\lambda)$. When the underlying TBPRF is instantiated by our NIKE TNIKE* (see Figure 12), we have $n = 0$.

Moreover, let $\mathcal{C}'_1$ represent the set of streaming word-RAM families with memory bounds m_u . Our construction in the BSM BSBPRF is defined in exactly the same way as in Figure 16 except that the underlying NCNIKE is replaced by BSMKE, which is a $\mathcal{C}'_1$ - n_p -NIKE with shared key space $\mathcal{K}'_\lambda$ and URS length n' satisfying 0-extendability, ϵ'_1 -correctness with $\epsilon'_1 = 2/\lambda$, and $(m_a, \epsilon', \epsilon'_2)$ -security in the BSM for some constant n_p , some ℓ' , and some $n' = n'(\lambda)$. In the full BPRF case, the all- $\perp$ constraint is handled by returning the master secret key and evaluating directly; all nontrivial constraints are handled by the combining and extracting procedures in Figure 16.

Theorem A.8 TBPRF is a $\mathcal{C}_1$ -($n_p - 1$)-BPRF with ϵ_1 -correctness and $\mathcal{C}_2$ - ϵ_2 -security.

Theorem A.9 BSBPRF is a $\mathcal{C}'_1$ -($n_p - 1$)-BPRF with ϵ'_1 -correctness and $(m_a, \epsilon', \epsilon'_2)$ -security.

The proofs of Theorem A.8 and Theorem A.9 are very similar to the proof of Theorem A.7 and thus we omit the details.

A.4 Extendability of Our Multi-Party NIKES

We now argue that our multi-party NIKE in the bounded-parallel-time model satisfies restricted extendability, and our multi-party NIKES in the bounded-time model and bounded-storage model satisfy extendability.

We first recall that NCNIKE (see Figure 5) satisfies 0-correctness, TNIKE* = $\{\text{Gen}_\lambda^*, \text{Share}_\lambda^*\}$ (see Figure 12) satisfies $((1 - (q/\ell)^{n_p-1})^q + 2\ell n_p \lambda^k / R)$ -correctness, and BSMKE (see Figure 14) satisfies $2/\lambda$ -correctness. Then we have the following theorem.

Theorem A.10 NCNIKE (see Figure 5) satisfies 0-restricted extendability, TNIKE* = $\{\text{Gen}_\lambda^*, \text{Share}_\lambda^*\}$ (see Figure 12) satisfies $2\ell n_p \lambda^k / R$ -extendability, and BSMKE (see Figure 14) satisfies 0-extendability.

The abort probabilities are accounted for in the correctness guarantees: TNIKE* has correctness error

$$\left(1 - (q/\ell)^{n_p-1}\right)^q + 2\ell n_p \lambda^k / R,$$

and BSMKE has correctness error $2/\lambda$. We define the combining and extracting algorithm families $\{\text{NCCombine}_\lambda, \text{NCExtract}_\lambda\}$, $\{\text{TCombine}_\lambda, \text{TExtract}_\lambda\}$, $\{\text{BSMCombine}_\lambda, \text{BSMExtract}_\lambda\}$ for NCNIKE, TNIKE*,

and BSMKE respectively as in Figure 17. Then Theorem A.10 follows immediately from the analysis of complexity and the definitions of the combining and extracting algorithms for NCNIKE, TNIKE*, and BSMKE. Indeed, one can easily see that for each scheme, the combining and extracting algorithms are exactly the intermediate steps of the sharing algorithm. The abort probabilities are exactly those established by the correctness analyses in Theorems 3.8, 4.8 and 5.9.

For BSMKE, a local key is represented together with the compact public hash descriptors that define its current index set. This adds only $O(n_p \log n)$ bits, since n_p is constant, and does not affect the asymptotic parameters.

NCCombine $_{\lambda}$ (localkey, $\{\mathbf{pk}_i\}_{i \in \mathcal{V}}$):
 Parse localkey = $\mathbf{R}$, $\{\mathbf{pk}_i\}_{i \in \mathcal{V}} = \{\{P_j\}_{j \in [t_1, s_1-1]} \cup \{P_j\}_{j \in [s_2+1, t_2]}\}$
 Return localkey' = $(\prod_{j=t_1}^{s_1-1} \mathbf{P}_j^{\top}) \mathbf{R} (\prod_{j=s_2+1}^{t_2} \mathbf{P}_j)$

NCExtract $_{\lambda}$ (localkey, $\mathbf{pk}_{n_p}$):
 Return key, which is the bottom-right bit of localkey

TCombine $_{\lambda}$ (localkey, $\{\mathbf{pk}_j\}_{j \in \mathcal{V}}$):
 Parse localkey = $\mathcal{S}$ and $\{\mathbf{pk}_j = ((I_j^t)_{t \in [\ell]}, \mathbf{v}_j)\}_{j \in \mathcal{V}}$
 For $j \in \mathcal{V}$, $\mathcal{T} = \emptyset$
 For every $s \in \mathcal{S}$, check by brute forcing whether I_j^s has one solution
 If the check passes, $\mathcal{T} = \mathcal{T} \cup \{s\}$
 $\mathcal{S} = \mathcal{S} \cap \mathcal{T}$
 Return localkey = $\mathcal{S}$

TExtract $_{\lambda}$ (localkey, $\mathbf{pk}_{n_p}$):
 Parse localkey = $\mathcal{S}$ and $\mathbf{pk}_{n_p} = ((I_{n_p}^j)_{j \in [\ell]}, \mathbf{v}_{n_p})$
 If $|\mathcal{S}| < 1$ abort
 Else return key = $(\bigoplus_{s \in \mathcal{S}} s) \cdot \mathbf{v}_{n_p}$

BSMCombine $_{\lambda}$ (localkey, $\{\mathbf{pk}_j\}_{j \in \mathcal{V}}$):
 Parse localkey = $(D, \text{urs}_{\mathcal{S}})$, where D is a list of hash functions and $\mathcal{S}$ is the intersection of the index sets defined by D
 Parse $\{\mathbf{pk}_j = (h_j, g_j)\}_{j \in \mathcal{V}}$
 Let $D' = D \cup \{h_j : j \in \mathcal{V}\}$
 Let $\mathcal{S}' = \mathcal{S} \cap \bigcap_{j \in \mathcal{V}} \mathcal{S}_j$, where $\mathcal{S}_j = \{h_j(a) : a \in [q]\}$
 Recompute $\mathcal{S}$ and $\mathcal{S}'$ from D and $(h_j)_{j \in \mathcal{V}}$ in a streaming manner, and remove from $\text{urs}_{\mathcal{S}}$ all bits whose indices are not in $\mathcal{S}'$
 Return localkey' = $(D', \text{urs}_{\mathcal{S}'})$

BSMExtract $_{\lambda}$ (localkey, $\mathbf{pk}_{n_p}$):
 Parse localkey = $(D, \text{urs}_{\mathcal{S}})$, recomputing $\mathcal{S}$ from D , and parse $\mathbf{pk}_{n_p} = (h_{n_p}, g_{n_p})$
 For each $j \in [q]$, let $u_j = h_{n_p}(j)$ if $h_{n_p}(j) \in \mathcal{S}$, and let $u_j = \perp$ otherwise
 Let $\mathbf{s}$ be obtained from $(u_j)_{j \in [q]}$ by removing all $\perp$'s while preserving the order of j
 Abort the whole protocol if $|\mathbf{s}| < \lambda$
 Let $\mathbf{s}_{\lambda}$ be the first λ entries of $\mathbf{s}$
 Return key = $g_{n_p}(\text{urs}_{\mathbf{s}_{\lambda}})$

Figure 17: Constructions of $\{\text{NCCombine}_{\lambda}, \text{NCExtract}_{\lambda}\}$, $\{\text{TCombine}_{\lambda}, \text{TExtract}_{\lambda}\}$, and $\{\text{BSMCombine}_{\lambda}, \text{BSMExtract}_{\lambda}\}$. Recall that for any set $\mathcal{S}$, $\text{urs}_{\mathcal{S}}$ denotes the sequence of all bits in urs with indices (ordered lexicographically) in $\mathcal{S}$; for any sequence $\mathbf{s}$, $\text{urs}_{\mathbf{s}}$ follows the order of $\mathbf{s}$.